

1. Introduction

1.1 Background

Student dropout is an important challenge in higher education because students may discontinue their studies before completing their programmes. Identifying students who are at risk of dropping out can help educational institutions understand student progression and provide appropriate support.

Machine Learning can be used to analyse student-related information and identify patterns associated with different academic outcomes. Instead of relying only on individual academic indicators, a Machine Learning model can consider multiple academic, demographic, social, and economic factors simultaneously to classify students according to their expected academic outcome.

This project focuses on developing a Machine Learning-based student outcome prediction system using the UCI Machine Learning Repository's *Predict Students' Dropout and Academic Success* dataset. The dataset contains information about students' academic background, demographic characteristics, socioeconomic factors, and academic performance during the first and second semesters. The prediction task is a three-class classification problem consisting of **Dropout**, **Enrolled**, and **Graduate** outcomes.

1.2 Problem Statement

Traditional identification of students who may be at risk of dropping out can be difficult when many different factors contribute to academic progression. A student's final outcome may be associated with academic performance, financial circumstances, previous educational background, and other characteristics.

The problem addressed in this project is therefore to develop a Machine Learning classification model capable of predicting a student's final academic outcome as **Dropout**, **Enrolled**, or **Graduate** using the available student information.

The project defines the prediction point as **after the completion of the second semester**. Therefore, information from both the first and second semesters is considered available to the prediction system. The final outcome represents the student's status at the end of the normal duration of the course.

1.3 Project Motivation

The motivation for this project is to demonstrate how Machine Learning can be applied to a practical educational problem while following a complete Machine Learning development process.

Rather than developing a model based only on a single academic variable, the project investigates relationships among multiple types of student information. Feature engineering is also used to derive additional indicators that represent academic progression and financial stress, allowing the models to use information that is more directly meaningful to the prediction task.

The project also aims to demonstrate the transition from a Machine Learning experiment to a usable software system by integrating the selected model into a full-stack application.

1.4 Project Objectives

The main objectives of the project are:

1. To investigate a real-world student outcome prediction problem using Machine Learning.
2. To obtain and analyse a suitable publicly available dataset.
3. To perform exploratory data analysis and identify important characteristics and relationships within the data.
4. To preprocess the dataset appropriately for Machine Learning.
5. To develop meaningful engineered features related to student academic performance and financial circumstances.
6. To develop and compare multiple Machine Learning classification models.
7. To evaluate the models using appropriate classification metrics rather than relying only on accuracy.
8. To select the most suitable model based on its performance.
9. To integrate the selected Machine Learning model into a full-stack application through a backend REST API.

1.5 Proposed Solution

The proposed solution consists of a Machine Learning-based prediction system that accepts student information, applies the same feature engineering and preprocessing procedures used during model development, and uses the trained model to predict one of three academic outcomes: **Dropout, Enrolled, or Graduate**.

The Machine Learning component is developed and evaluated using the selected dataset. Multiple models are compared before selecting the final model. The selected model is then intended to be integrated into a backend REST API, allowing a frontend application to submit student information and receive the predicted outcome.

The overall solution therefore combines Machine Learning development with software application development rather than treating the trained model as an isolated experiment.

1.6 Scope of the Project

The scope of this project covers the complete Machine Learning development process from dataset analysis to model selection and application integration.

The Machine Learning scope includes exploratory data analysis, data-quality investigation, train-test splitting, feature engineering, categorical encoding, numerical feature scaling, model development, hyperparameter tuning, and model evaluation.

The application scope includes exposing the trained model through a backend REST API and providing a frontend interface through which users can submit the required student information and receive a prediction.

The project focuses on predicting the three outcome categories provided by the selected dataset. It does not attempt to establish causal relationships between individual student characteristics and dropout, nor does a model prediction guarantee a student's actual future academic outcome.

2. Dataset and Project Methodology

2.1 Machine Learning Problem Definition

This project is formulated as a **multi-class classification problem**. The objective is to predict the final academic outcome of a student using information describing the student's academic background, demographic characteristics, socioeconomic circumstances, and academic performance.

The target variable contains three possible classes:

- **Dropout** – the student discontinued the programme.
- **Enrolled** – the student remained enrolled without completing the programme at the time represented by the target.
- **Graduate** – the student successfully completed the programme.

The UCI dataset defines these three categories as the final target outcomes at the end of the normal duration of the course.

2.2 Dataset Source

The dataset used in this project is **Predict Students' Dropout and Academic Success**, obtained from the **UCI Machine Learning Repository**.

The dataset was created using information from a higher education institution and contains records from students enrolled in different undergraduate programmes. According to the official UCI documentation, the dataset contains **4,424 student records and 36 features**. It is a tabular classification dataset and contains information available at student enrolment together with academic performance information from the first and second semesters.

The official dataset citation is:

Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2021). *Predict Students' Dropout and Academic Success*. UCI Machine Learning Repository. DOI: 10.24432/C5MC89.

2.3 Dataset Overview

Each row in the dataset represents an individual student. The dataset contains several different categories of information, allowing the prediction problem to be examined from multiple perspectives.

The main groups of information include:

- **Demographic information**, such as age, gender, nationality, and marital status.
- **Application and educational background**, such as application mode, course, previous qualification, and previous qualification grade.
- **Family and socioeconomic information**, including parents' qualifications and occupations.
- **Student status information**, such as debtor status, scholarship status, tuition-fee status, and international-student status.
- **First-semester academic information**, including enrolled, evaluated, approved, and credited curricular units and semester grades.
- **Second-semester academic information**, containing corresponding academic performance measures.
- **Economic indicators**, including unemployment rate, inflation rate, and GDP.

The official UCI feature documentation identifies the dataset as containing 36 features with no missing values.

2.4 Target Variable

The target variable used in the project is **Target**.

The original target is categorical and contains three classes:

Target class	Meaning
Dropout	Student dropped out
Enrolled	Student remained enrolled
Graduate	Student graduated

The target variable is separated from the input features before the Machine Learning workflow. The resulting feature matrix is represented by **X**, while the target variable is represented by **y**.

The three-class structure is important because the project is not simply predicting whether a student will drop out or not. Instead, the model must distinguish among **three different academic outcomes**.

2.5 Feature Types

The 36 input features were organised according to their role and data characteristics before preprocessing.

The feature groups used in the project were:

- **11 categorical features**
- **18 numerical features**
- **7 binary features**

Categorical variables represent coded categories such as course, application mode, previous qualification, and parental qualification or occupation.

Numerical variables include continuous or count-based measurements such as grades, age, curricular-unit counts, and economic indicators.

Binary variables represent two-state characteristics such as debtor status, tuition-fee status, scholarship status, and international status.

This grouping was important because different feature types require different preprocessing operations. Categorical variables were encoded using one-hot encoding, numerical variables were standardized, and binary variables were passed through without one-hot expansion.

2.6 Overall Machine Learning Methodology

The project follows a structured Machine Learning workflow:

Dataset Acquisition → Exploratory Data Analysis → Data Quality Checks → Train/Test Split → Feature Engineering → Data Preprocessing → Model Development → Hyperparameter Tuning → Model Evaluation → Model Selection → Application Integration

The dataset is first examined to understand its structure, target distribution, missing values, duplicates, possible invalid values, statistical characteristics, and relationships between features and the target.

After exploratory analysis, the data is divided into training and testing sets. An **80:20 split** is used, with stratification applied to preserve the approximate class distribution in both sets. The UCI documentation also describes an 80% training and 20% testing split for the dataset.

Feature engineering is then performed using information available at the defined prediction point. Five additional features are created:

1. `Sem1_approval_rate`
2. `Sem2_approval_rate`
3. `Grade_change`
4. `Financial_stress`

5. `Approval_rate_change`

These features are intended to represent academic performance, academic progression, and a combination of financial-risk indicators more directly than some of the original variables alone.

Following feature engineering, categorical variables are encoded using `OneHotEncoder`, numerical variables are standardized using `StandardScaler`, and binary variables are retained using passthrough processing. A `ColumnTransformer` is used to apply these transformations according to the feature groups.

Three classification algorithms are then developed and compared:

- Logistic Regression
- Random Forest
- Gradient Boosting

Hyperparameter tuning is performed using cross-validation, and the final model is selected based on the comparative evaluation results. The selected model is subsequently intended to be integrated into the full-stack application through a backend REST API.

The detailed exploratory analysis and data-quality investigation are presented in **Section 3**, followed by the feature engineering and preprocessing stages in the subsequent sections.

3. Exploratory Data Analysis

3.1 Dataset Overview

The dataset used for this project is the **Predict Students' Dropout and Academic Success** dataset obtained from the UCI Machine Learning Repository. The dataset contains information about students' demographic characteristics, admission information, financial status, and academic performance during the first and second semesters.

The dataset contains **4,424 student records**, with **36 input features** and **one target variable**. Each row represents an individual student, while each feature describes a characteristic or measurement associated with that student.

The target variable represents the student's eventual academic outcome and contains three classes:

- **Graduate**
- **Dropout**
- **Enrolled**

The prediction problem is therefore formulated as a **three class classification problem**.

The feature descriptions were inspected using the metadata provided with the UCI dataset. This was particularly important because many of the dataset's features are represented using numerical codes. For example, variables such as *Course*, *Application mode*, *Previous qualification*, and *Marital Status* contain numerical category codes rather than continuous numerical measurements.

3.2 Target Variable Distribution

The distribution of the target variable was examined before model development to understand the balance between the three outcome classes.

The dataset contains:

Outcome	Number of Students	Approx. Percentage
Graduate	2,209	49.9%
Dropout	1,421	32.1%
Enrolled	794	18.0%
Total	4,424	100%

The **Graduate** class is the largest class, representing approximately half of the dataset, while **Enrolled** is the smallest class.

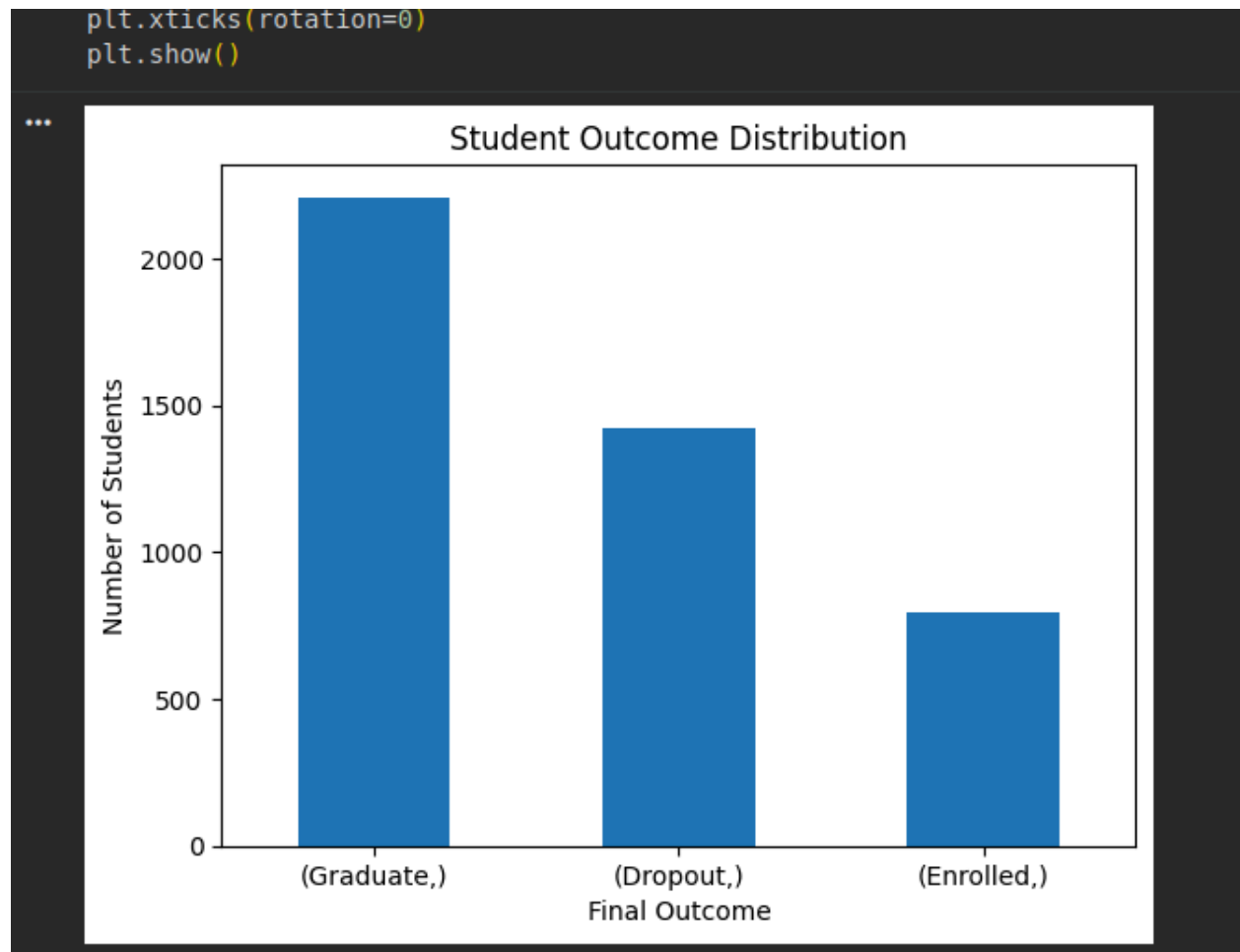
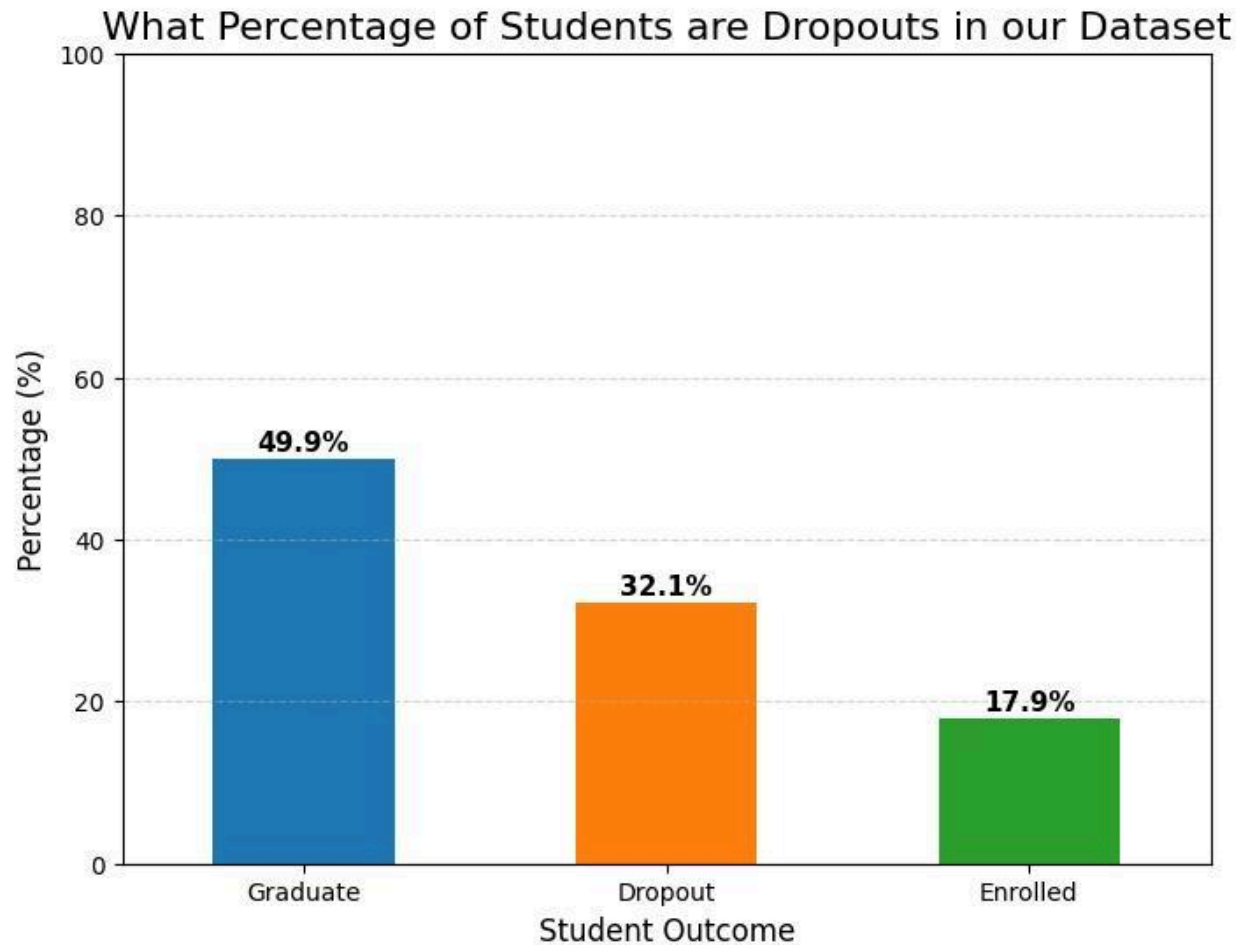


Figure 1. Distribution of students across the three academic outcome classes.



The classes are therefore not perfectly balanced. However, the dataset is not extremely imbalanced. Because the class frequencies differ, model evaluation should not rely exclusively on accuracy. Precision, recall, F1-score, and the confusion matrix should also be considered to determine whether the models perform adequately across all three classes.

3.3 Duplicate Record Analysis

Exact duplicate records were checked using the feature dataset.

The duplicate check returned:

0 duplicate records.

This indicates that there were no exact duplicate rows among the 36 feature columns. Therefore, no duplicate records needed to be removed during the initial data-quality analysis.

It should be noted that this check identifies **exact duplicates only**. It does not determine whether two different students happen to have identical feature values.

3.4 Missing Value Analysis

Missing values were examined across all 36 input features.

The analysis found **no missing values** in any of the feature columns. Therefore, missing-value imputation was not required for this dataset.

This is an important finding because missing-value treatment would otherwise be required before applying many machine learning algorithms.

Since no missing values were found, missing-value imputation was **not artificially applied** as a preprocessing technique.

3.5 Descriptive Statistical Analysis

Descriptive statistics were calculated for the dataset to examine the ranges and distributions of the numerical representations of the features.

The analysis showed that the dataset contains a mixture of:

- Continuous numerical variables
- Binary variables
- Categorical variables represented using numerical codes
- Academic performance measurements
- Curricular-unit counts
- Macroeconomic indicators

An important observation is that a numerical data type does **not necessarily mean that a feature is continuous**.

For example, **Course** contains numerical codes such as **33**, **171**, **8014**, and **9500**, but these values represent different academic programs rather than quantities where the difference between 33 and 171 has mathematical meaning.

Similarly, variables such as **Previous qualification**, **Application mode**, and **Marital Status** are categorical variables represented by numerical codes.

Therefore, the data types were considered according to the **meaning of each feature**, rather than simply treating every integer valued column as a continuous numerical variable.

3.6 Data Validity and Quality Checks

Several checks were performed to determine whether the values in the dataset were within reasonable and documented ranges.

Grade ranges

The following variables were examined:

- Previous qualification (grade)
- Admission grade

The observed values were within the documented grade range of the dataset. No values outside the expected range were identified.

The curricular-unit semester grades were also inspected. The observed ranges were:

- First-semester grade: **0 to 18.875**
- Second-semester grade: **0 to approximately 18.571**

These values are within the documented 0–20 grading scale.

Therefore, no invalid grade values were identified.

Binary feature validation

The following binary variables were inspected:

- Displaced
- Educational special needs
- Debtor
- Tuition fees up to date
- Gender
- Scholarship holder
- International

The observed values for these variables were **0 and 1**, matching their definitions in the dataset metadata.

Therefore, no unexpected values were identified in these binary features.

Categorical feature validation

Several categorical-coded variables were examined using their frequency distributions, including:

- Marital Status
- Course
- Application mode
- Previous qualification

The observed category codes were consistent with the categories described in the dataset metadata.

Some categories occur very rarely. These rare categories were not automatically considered errors because their presence is consistent with legitimate categories defined by the dataset.

3.7 Logical Consistency Checks

Logical relationships between curricular unit variables were also examined.

For both semesters, the following condition was checked:

Number of approved curricular units > number of enrolled curricular units

The result was **zero violations for both semesters**.

This indicates that the number of approved units does not exceed the number of enrolled units.

The number of curricular units without evaluations was also checked against the number of enrolled units. No cases were found where the number without evaluations exceeded the number enrolled.

The relationship between **evaluations** and **enrolled units** was also examined. In the first semester, there were **2,792 records where the number of evaluations was greater than the number of enrolled curricular units**.

This was not treated as invalid data because the dataset defines *evaluations* separately from *enrolled curricular units*. Therefore, an evaluation count can exceed the number of enrolled units, and this observation alone does not establish a data-quality error.

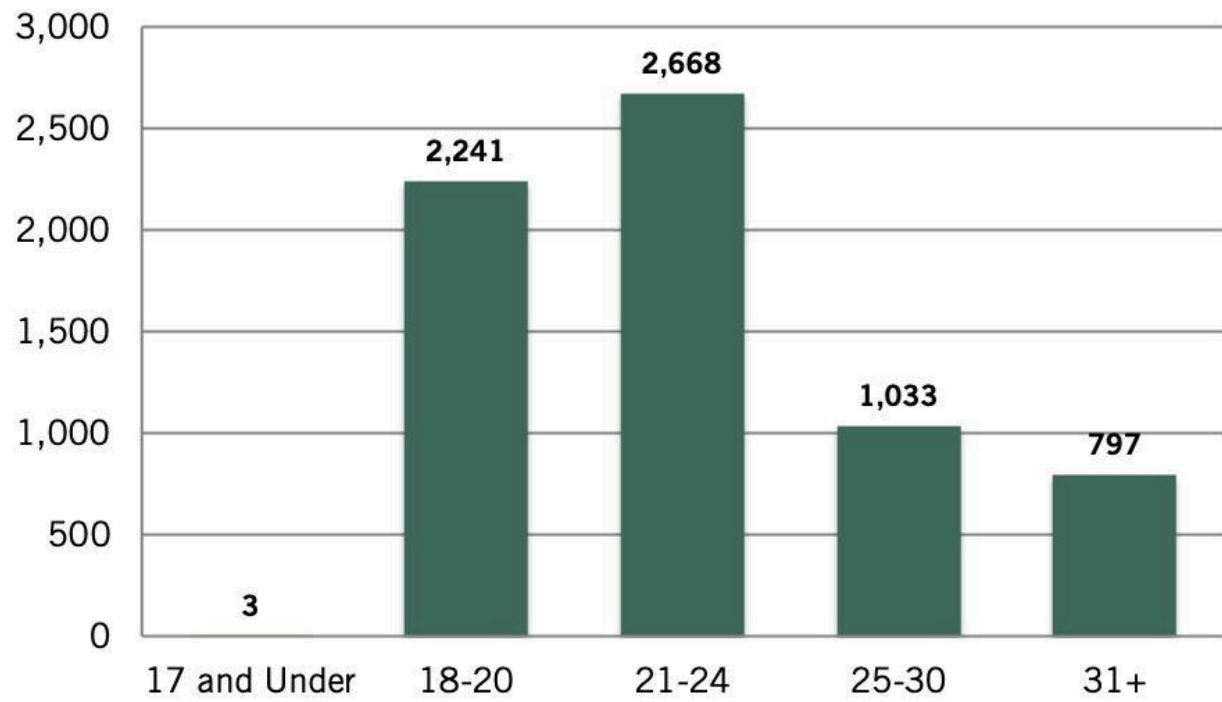
3.8 Age Distribution

The age of students at enrollment was examined to understand the demographic distribution.

The observed age range was:

- **Minimum:** 17
- **Maximum:** 70
- **Mean:** approximately 23.27
- **Median:** 20

Most students were concentrated in the late-teen and early-adult age range, while older students were less common.



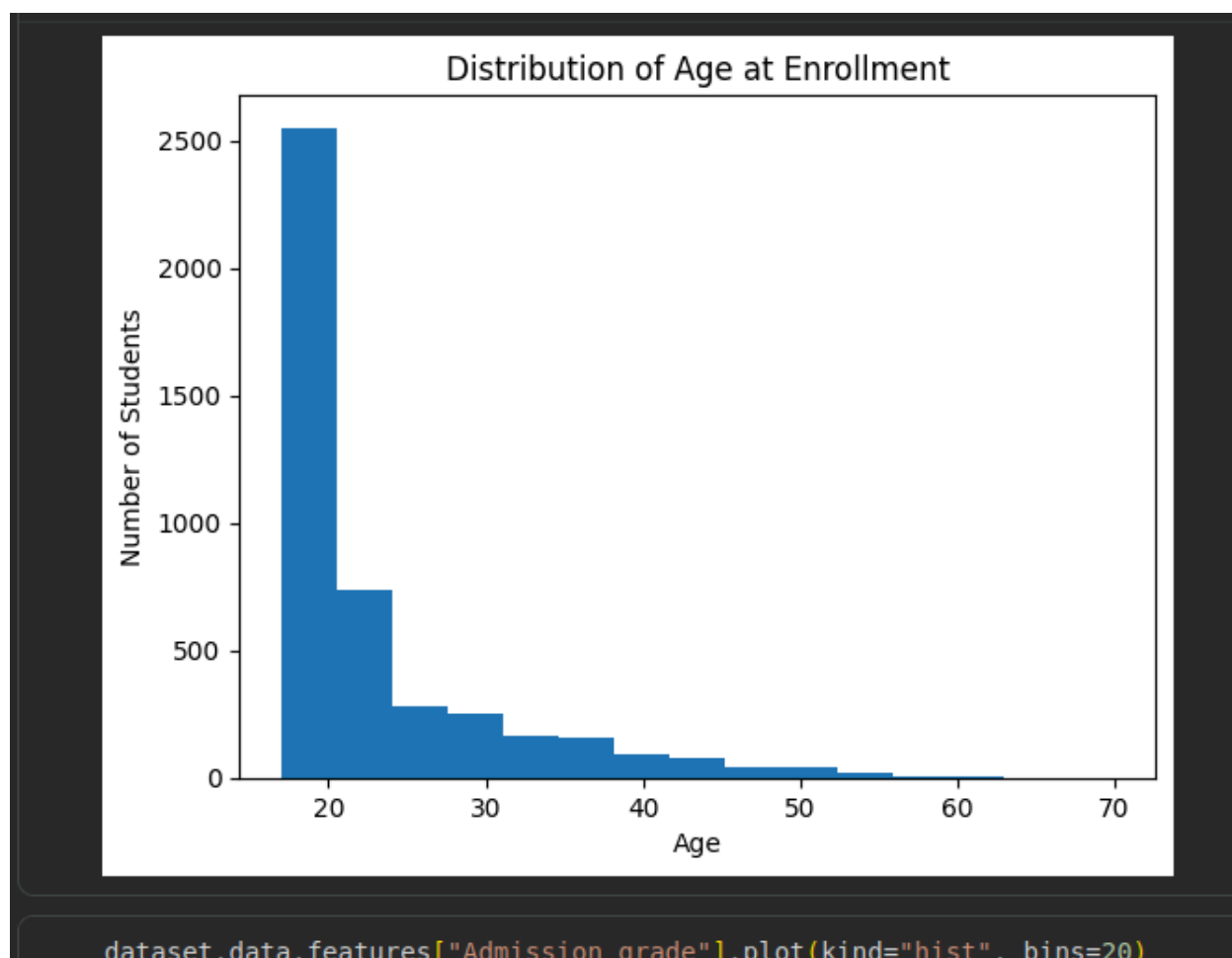


Figure 2. Distribution of student age at enrollment.

Although ages above the main concentration of students are relatively uncommon, they were not automatically treated as invalid because adult and older students can legitimately enroll in higher education.

3.9 Admission Grade Distribution

The admission-grade distribution was examined to understand the range and concentration of students' entry grades.

The observed admission grades ranged from **95 to 190**, with:

- Mean: approximately **126.98**
- Median: approximately **126.10**

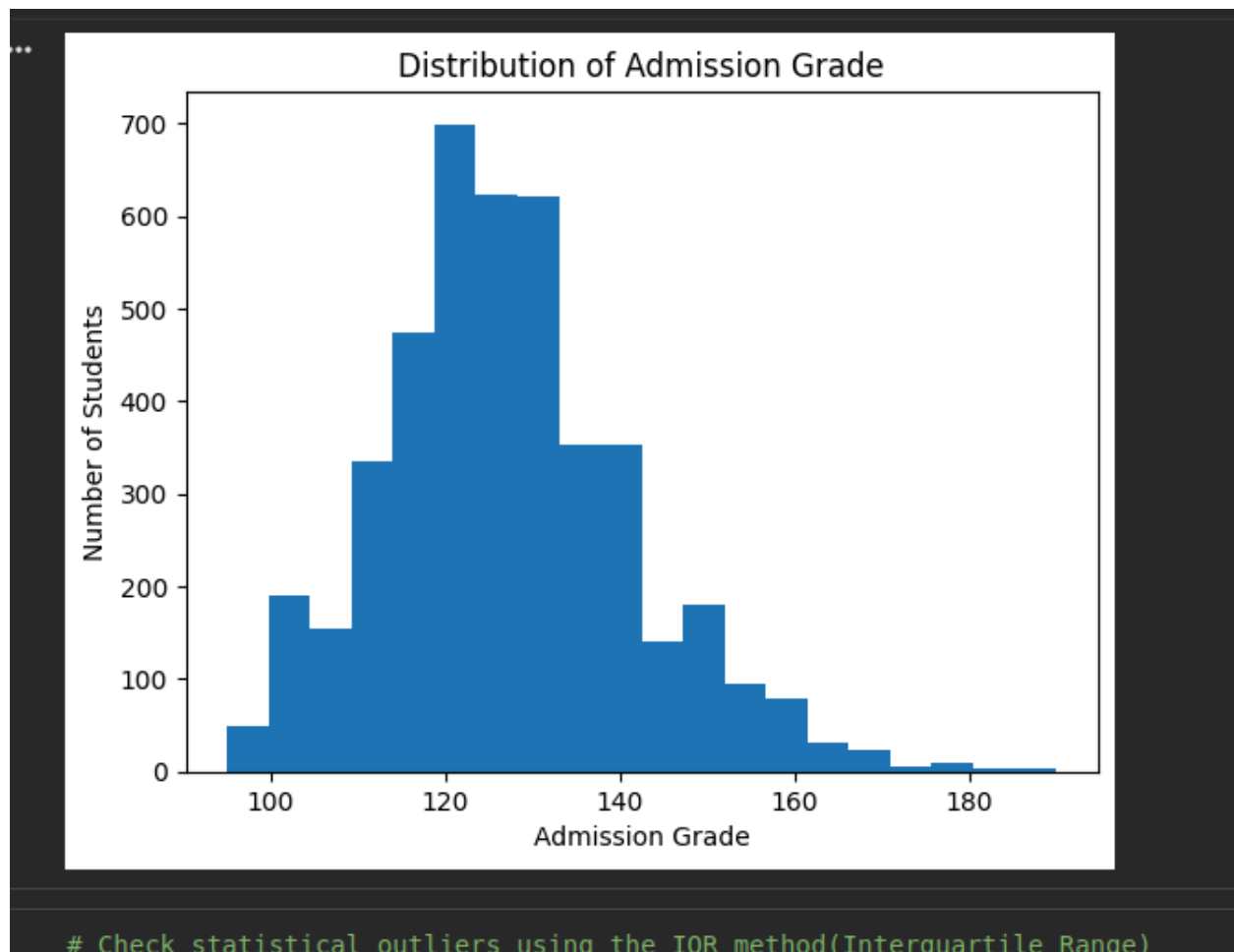


Figure 3. Distribution of students' admission grades.

The distribution provides an overview of the academic performance of students at the point of admission and provides useful context before examining relationships between admission characteristics and eventual academic outcomes.

3.10 Outlier Analysis

Statistical outlier analysis was performed using the **Interquartile Range (IQR) method**.

For each selected numerical variable, the first quartile (Q1), third quartile (Q3), and IQR were calculated:

$$\text{IQR} = \text{Q3} - \text{Q1}$$

Observations below:

$$\text{Q1} - 1.5 \times \text{IQR}$$

or above:

$$\text{Q3} + 1.5 \times \text{IQR}$$

were classified as statistical outliers.

The analysis was performed on five variables:

Feature	Statistical Outliers
Age at enrollment	441
Admission grade	86
Previous qualification (grade)	179
1st-semester grade	726
2nd-semester grade	877

These observations were **not removed** solely because they were classified as statistical outliers.

This is because being an IQR outlier does not necessarily mean that a value is incorrect. For example, an older student may legitimately be enrolled at university, and a student's academic grade can legitimately be unusually low or high relative to the majority of students while still remaining within the valid grading range.

Therefore, the statistical outlier analysis was used to identify unusual observations rather than automatically treating them as errors.

3.11 Feature - Target Relationship Analysis

Selected feature–target relationships were examined to determine whether important student characteristics differed between the three eventual outcomes.

The analysis focused on representative variables from several categories rather than manually testing every individual feature.

Academic performance

The average first-semester grade differed considerably between the outcome groups:

Outcome	Average 1st-Semester Grade
Dropout	7.26
Enrolled	11.13
Graduate	12.64

This indicates a strong association between first-semester academic performance and eventual outcome.

The average number of approved first-semester curricular units also differed:

Outcome	Average 1st-Semester Approved Units
Dropout	2.55
Enrolled	4.32

Graduate	6.23
----------	------

A similar pattern was observed for second-semester approved curricular units:

Outcome	Average 2nd-Semester Approved Units
Dropout	1.94
Enrolled	4.06
Graduate	6.18

The second-semester average grade showed a particularly clear separation:

Outcome	Average 2nd-Semester Grade
Dropout	5.90
Enrolled	11.12
Graduate	12.70

These results indicate that students' academic performance is strongly associated with their eventual academic outcome.

Important: these results demonstrate **association, not causation**.

Admission characteristics

Admission grade showed a smaller difference between outcome groups:

Outcome	Average Admission Grade
---------	-------------------------

Dropout	124.96
Enrolled	125.53
Graduate	128.79

Graduate students had the highest average admission grade, but the separation between the groups was relatively small compared with the differences observed in semester performance.

Previous qualification grade showed a similar pattern:

Outcome	Average Previous Qualification Grade
Dropout	131.11
Enrolled	131.21
Graduate	134.08

This suggests that pre-admission academic performance has some association with eventual outcome, but the separation is weaker than that observed for actual university semester performance.

Financial status

The relationship between **tuition fee status** and eventual outcome was also examined.

The percentage of students whose tuition fees were up to date was:

Outcome	Fees Up to Date	Fees Not Up to Date
Dropout	67.84%	32.16%
Enrolled	94.71%	5.29%

Graduate	98.69%	1.31%
----------	--------	-------

This shows a strong association between tuition fee status and eventual outcome in this dataset.

The **Debtor** variable showed a similar pattern:

Outcome	Non-Debtor	Debtor
Dropout	78.04%	21.96%
Enrolled	88.67%	11.33%
Graduate	95.43%	4.57%

Again, these results indicate **association rather than causation**. The analysis does not establish that financial status causes a student to drop out.

Application Order

Application order was investigated because the dataset records whether the selected course was the student's first or later application choice. A derived **First_choice** feature was tested to determine whether selecting the first choice was meaningfully associated with the target.

However, only one training instance was identified as a first-choice applicant. Although this single observation belonged to the Graduate class, the sample size was too small to support a meaningful conclusion.

Therefore, the derived **First_choice** feature was rejected. The original **Application order** feature was retained for further modelling.

Economic Context

The macroeconomic variables : unemployment rate, inflation rate, and GDP ,were examined to determine whether an overall economic-stress feature should be created.

The individual variables showed relatively weak and inconsistent relationships with the target. GDP showed some directional differences between groups, but the evidence was not strong enough to justify combining the three variables into an arbitrary economic-stress index.

Therefore, no composite economic-stress feature was created. The original unemployment rate, inflation rate, and GDP variables were retained as separate input features.

3.12 Data Leakage Analysis

Data leakage was considered based on the defined prediction point.

For this project, the prediction is made **after the student has completed the second semester**. Therefore, information from the first and second semesters can legitimately be used as model input because it is available at the prediction point.

The dataset metadata was inspected to identify whether any input feature directly represented the final academic outcome or contained information that would only become available after the outcome.

The **Target** variable represents the final outcome and is therefore separated from the input features during model development.

No obvious feature was identified that directly duplicates or reveals the target.

The second-semester academic features were **not considered leakage** under the project's prediction definition because they represent information available by the end of the second semester.

This distinction is important because leakage depends on **when the prediction is supposed to be made**. If the prediction were instead intended to be made at the beginning of the first semester, second-semester information would constitute leakage.

3.13 EDA Summary

The exploratory data analysis provided several important findings about the dataset.

First, the dataset contains **4,424 students and 36 input features**, with three outcome classes: Graduate, Dropout, and Enrolled. The target classes are not equally distributed, with Graduate being the largest class and Enrolled being the smallest.

No exact duplicate feature records or missing values were identified. The binary and categorical-coded features were checked against their documented values, and no obvious invalid category values were identified.

The numerical range checks showed that the examined grades and curricular-unit counts remained within valid ranges. Logical consistency checks also found no cases where approved curricular units exceeded enrolled units.

The IQR analysis identified statistical outliers in several numerical variables. However, these observations were retained because statistical unusualness does not necessarily indicate invalid data.

The feature–target analysis showed particularly strong differences in **semester academic performance** between the three outcome groups. Financial-status variables such as tuition-fee status and debtor status also showed clear associations with eventual outcome, while admission-related grades showed comparatively weaker separation.

Finally, the dataset was examined for potential data leakage. Based on the project's prediction point of **after the second semester**, the available semester-performance information can legitimately be used for prediction, and no obvious feature directly revealing the target was identified.

4. Feature Engineering and Data Transformation

Feature engineering was performed to create additional information from the original variables that could better represent students' academic progress and financial circumstances. The original dataset contains 36 features, including categorical, numerical, and binary variables. Since the prediction task uses information available up to the end of the second semester, the first- and second-semester academic variables were considered valid predictors rather than data leakage. The UCI dataset explicitly contains academic performance information from both semesters for predicting the final student outcome.

The feature engineering and transformation process consisted of creating meaningful derived features, encoding categorical variables, standardizing numerical variables, and retaining binary variables in their original representation.

4.1 Semester 1 Approval Rate

The number of approved curricular units alone does not account for differences in the number of units a student enrolled in. Therefore, an approval rate was created to represent the proportion of enrolled units that were successfully approved:

Semester 1 Approval Rate = Approved Units / Enrolled Units

A total of 148 training instances had zero enrolled units in the first semester. Inspection showed that these students also had zero approved units, evaluations, grades, and units without evaluations. This represented a consistent no-activity pattern rather than an invalid combination of values.

Because approval rate is undefined when both approved and enrolled units are zero, these cases were assigned a value of 0 to represent the absence of measurable academic activity. This value should not be interpreted as a literal 0% approval rate.

4.2 Semester 2 Approval Rate

The same transformation was applied to the second semester:

Semester 2 Approval Rate = Approved Units / Enrolled Units

The same 148 training instances had zero enrolled units in the second semester, and inspection confirmed the same consistent zero-activity pattern. These cases were therefore handled using the same approach as the first-semester approval rate.

The resulting approval-rate features are bounded between 0 and 1 and provide a normalized representation of academic achievement relative to each student's enrolled workload.

4.3 Grade Change Between Semesters

To represent changes in academic performance over time, the difference between second-semester and first-semester average grades was calculated:

Grade Change = Second Semester Grade – First Semester Grade

Positive values indicate an improvement in average grade, negative values indicate a decrease, and zero indicates no change.

The mean grade change in the training set was approximately -0.42, indicating a small average decrease from the first to the second semester. The median was 0, showing that many students had little or no change in their average grade.

4.4 Financial Stress

Exploratory analysis showed a strong relationship between financial status and the final student outcome. In particular, being a debtor while also having tuition fees that were not up to date was strongly associated with dropout.

Therefore, a combined binary feature called `Financial_stress` was created:

Financial Stress = 1 if Debtor = 1 AND Tuition Fees Up to Date = 0; otherwise 0

In the training set, 196 students were classified as experiencing this combined financial condition.

Among these students, approximately 88.3% belonged to the Dropout class, compared with approximately 28.8% among students without this condition. This provided sufficient evidence to retain the derived feature.

4.5 Approval Rate Change

To capture academic progress between semesters, the change in approval rate was calculated:

Approval Rate Change = Semester 2 Approval Rate – Semester 1 Approval Rate

Negative values represent a decrease in the proportion of successfully approved units, while positive values represent an improvement.

Exploratory analysis showed a meaningful relationship with the target. Students with a large decrease in approval rate had a 56.7% Dropout proportion, while students with no change had a 60.3% Graduate proportion. Students with a small increase had a 51.8% Graduate proportion.

Therefore, the continuous `Approval_rate_change` feature was retained.

4.6 Categorical Feature Encoding

The dataset contains several variables represented by integer codes, such as application mode, course, previous qualification, nationality, and parental qualifications. Although these variables are stored numerically, their codes represent categories rather than quantities with meaningful numerical distances. For example, the UCI documentation defines **Application mode** as different admission pathways and **Course** as different degree programmes.

Therefore, the categorical variables were transformed using **One-Hot Encoding**.

Each category was converted into a separate binary feature. This prevents the machine-learning model from incorrectly interpreting category codes as continuous numerical values.

The encoder was fitted using only the training data:

```
OneHotEncoder(handle_unknown="ignore")
```

The **handle_unknown="ignore"** option allows the transformation to continue if a category appears in the test set or future data that was not present in the training set.

4.7 Numerical Feature Standardization

The numerical features have different scales and ranges. For example, grades are measured on a different scale from curricular-unit counts, age, and macroeconomic variables.

Therefore, numerical features were standardized using **StandardScaler**. Standardization transforms each numerical feature using the mean and standard deviation learned from the training data:

Standardized value = (value – training mean) / training standard deviation

The scaler was fitted only on the training set and the same learned parameters were then applied to the test set. This prevents information from the test set from influencing the transformation process.

Standardization is particularly important for models such as Logistic Regression because it places numerical features on comparable scales.

4.8 Binary Features

The dataset contains several naturally binary variables, including `Debtor`, `Tuition fees up to date`, `Gender`, `Scholarship holder`, and `International`.

These variables already use binary representations, so they were retained using a passthrough transformation rather than applying one-hot encoding or standardization.

The newly created `Financial_stress` feature is also binary and was treated in the same way.

4.9 Combined Transformation

A `ColumnTransformer` was used to apply the appropriate transformation to each feature group:

- **Categorical features:** One-Hot Encoding
- **Numerical features:** Standardization
- **Binary features:** Passed through unchanged

This produced a consistent model-ready representation while keeping the original training and test DataFrames unchanged.

The original dataset contained 36 input features. After feature engineering, five additional features were retained, resulting in 41 input columns before categorical encoding. One-hot encoding then expanded the categorical variables into multiple binary columns.

The preprocessing transformations were fitted using the training data only and then applied to both the training and test data. This preserves the separation established during the train-test split and prevents information from the test set from being used during preprocessing.

5. Data Preprocessing

Following the exploratory data analysis and feature-engineering stages, the dataset was prepared for machine learning. The purpose of preprocessing was to represent each feature according to its actual meaning, place numerical variables on comparable scales, and ensure that the same transformation rules were applied consistently during model training and evaluation.

The dataset contains 36 original input features consisting of categorical, numerical, and binary variables. In addition, five engineered features were created during the feature-engineering stage. Therefore, two feature sets were maintained for the subsequent modelling stage:

- Baseline dataset: the original 36 input features.
- Engineered dataset: the original 36 features plus 5 engineered features, giving 41 input features.

Maintaining these two versions allows the effect of the engineered features to be evaluated experimentally rather than assuming that they improve model performance.

The preprocessing workflow therefore consisted of:

1. Separating the input features and target variable.

2. Classifying the features according to their actual meaning.
3. Splitting the data into training and testing sets.
4. Creating separate baseline and engineered feature sets.
5. Applying one-hot encoding to categorical features.
6. Standardizing numerical features.
7. Passing binary features through unchanged.
8. Combining these transformations using separate `ColumnTransformer` objects.
9. Fitting each preprocessor using training data only.
10. Applying the fitted preprocessing rules to both training and testing data.
11. Verifying that the resulting training and testing representations have matching dimensions.

5.1 Separation of Input Features and Target

The target variable in this project is `Target`, which represents the student's final academic outcome. It contains three possible classes:

- Dropout
- Enrolled
- Graduate

The remaining 36 variables were treated as the original input features.

The features were assigned to `X`, while the target was assigned to `y`. This separation ensures that the final outcome is not accidentally included among the variables supplied to the model.

The resulting dataset contained:

Component	Shape
Input features (<code>X</code>)	$4,424 \times 36$
Target (<code>y</code>)	4,424

Original input features	36
Target classes	3

The target distribution had already been examined during EDA. Because the three classes are not equally represented, stratification was used during the train/test split.

5.2 Feature Classification

Before preprocessing, every input feature was explicitly assigned to one of three groups based on its meaning rather than simply its storage data type.

Categorical Features

The baseline dataset contains 11 categorical features:

- Marital Status
- Application mode
- Application order
- Course
- Daytime/evening attendance
- Previous qualification
- Nationality
- Mother's qualification
- Father's qualification
- Mother's occupation
- Father's occupation

Although these variables are represented using numerical codes in the original dataset, the codes identify categories rather than numerical quantities.

For example, a **Course** code of one value is not mathematically greater than another course code. Treating such codes as continuous numerical measurements would therefore introduce an artificial ordering and distance between categories.

These features were consequently treated as categorical variables and one-hot encoded.

Numerical Features

The baseline dataset contains 18 numerical features:

- Previous qualification (grade)
- Admission grade
- Age at enrollment
- Curricular units 1st sem (credited)
- Curricular units 1st sem (enrolled)
- Curricular units 1st sem (evaluations)
- Curricular units 1st sem (approved)
- Curricular units 1st sem (grade)
- Curricular units 1st sem (without evaluations)
- Curricular units 2nd sem (credited)
- Curricular units 2nd sem (enrolled)
- Curricular units 2nd sem (evaluations)
- Curricular units 2nd sem (approved)
- Curricular units 2nd sem (grade)
- Curricular units 2nd sem (without evaluations)
- Unemployment rate
- Inflation rate
- GDP

These variables represent measurements or quantities for which numerical relationships are meaningful. They were therefore retained as numerical features and standardized.

Binary Features

The baseline dataset contains seven binary features:

- Displaced
- Educational special needs
- Debtor
- Tuition fees up to date
- Gender
- Scholarship holder
- International

These variables were already represented using 0/1 values. Therefore, they were passed through the preprocessing stage unchanged.

Feature Classification Summary

Feature type	Baseline features	Preprocessing
Categorical	11	One-hot encoding
Numerical	18	Standardization
Binary	7	Passed through unchanged
Total	36	

A completeness check confirmed that all 36 original features were assigned exactly one preprocessing role. There were no missing columns, unexpected columns, or duplicate assignments.

5.3 Training and Testing Split

The dataset was divided into training and testing subsets using an 80:20 split.

Dataset	Students	Percentage
Training	3,539	80%
Testing	885	20%

Total	4,424	100%
-------	-------	------

The split was performed using stratification based on the target variable. This maintains approximately the same proportions of Dropout, Enrolled, and Graduate students in both subsets.

A fixed random state was also used so that the split remains reproducible.

The training data is used to learn preprocessing parameters and train the models, while the testing data remains separate for the final evaluation.

Importantly, the preprocessing transformations were fitted only after this split. Therefore, information from the testing set was not used to learn the encoding categories or numerical scaling parameters.

5.4 Baseline and Engineered Feature Sets

After the train/test split, two separate versions of the input data were created.

Baseline Feature Set

The baseline version contains only the original 36 features.

Feature group	Number
Categorical	11
Numerical	18

Binary	7
Total	36

The baseline provides a reference against which the engineered version can later be compared.

Engineered Feature Set

The engineered version contains the same original 36 features plus five engineered features created during the feature-engineering stage:

- Sem1_approval_rate
- Sem2_approval_rate
- Grade_change
- Financial_stress
- Approval_rate_change

These five features were added according to their appropriate preprocessing roles.

Feature group	Number
Categorical	11
Numerical	22
Binary	8

Total	41
-------	----

The engineered feature grouping was also checked independently to ensure that all 41 columns were assigned exactly one preprocessing role.

This gives two controlled experiments:

Baseline: 36 original features

Engineered: 36 original + 5 engineered features = 41 features

The same preprocessing strategy is then applied separately to both datasets.

5.5 Categorical Encoding

Categorical features were transformed using `OneHotEncoder`.

One-hot encoding creates a separate binary feature for each category, allowing categorical variables to be represented numerically without imposing an artificial numerical ordering.

The encoder was configured with:

```
OneHotEncoder(handle_unknown="ignore")
```

The `handle_unknown="ignore"` setting allows the transformer to process a category encountered during testing or future prediction that was not present in the training data. Instead of producing an error, the unknown category is represented without creating a new feature column.

The encoder learns its category structure from the training data. The same learned structure is then used when transforming the testing data.

5.6 Numerical Feature Standardization

The numerical features were standardized using `StandardScaler`.

The numerical variables have different scales and units. For example, age is measured in years, curricular-unit counts are small integer quantities, grades use a different scale, and economic indicators such as GDP have substantially different magnitudes.

Standardization places these numerical variables onto a comparable scale by using the mean and standard deviation learned from the training data.

The important point is that these parameters are learned only from the training set. The testing data is not used to calculate a separate mean or standard deviation.

Therefore, the process is:

Training data



Learn mean and standard deviation



Transform training data



Use the SAME parameters



Transform testing data

This prevents information from the testing set from influencing the preprocessing stage.

5.7 Binary Feature Handling

The binary features were already represented using 0 and 1.

No additional encoding or scaling was applied to these variables. They were therefore passed through unchanged.

For the baseline dataset, seven binary features were passed through.

For the engineered dataset, the `Financial_stress` feature was also treated as binary, increasing the binary feature count to eight.

This preserves the existing interpretation of these variables while avoiding unnecessary transformations.

5.8 Separate Preprocessing for Baseline and Engineered Data

Because the project compares the original feature set against the engineered feature set, separate preprocessing objects were created for the two experiments.

The baseline preprocessor applies:

- One-hot encoding to 11 categorical features;
- standardization to 18 numerical features;
- passthrough to 7 binary features.

The engineered preprocessor applies:

- One-hot encoding to 11 categorical features;
- standardization to 22 numerical features;
- passthrough to 8 binary features.

This separation ensures that the two experiments remain independent and that the baseline representation does not accidentally contain the engineered features.

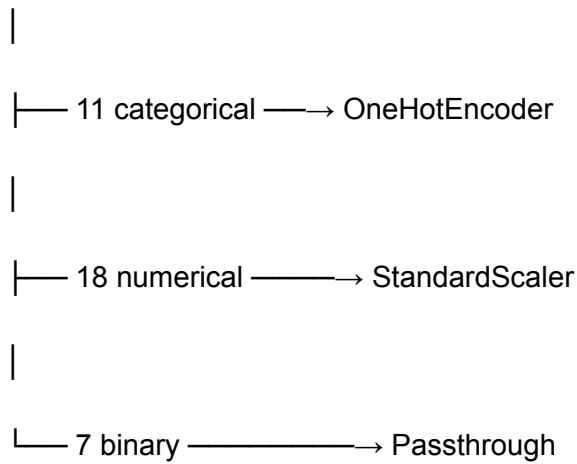
5.9 Combined Preprocessing Using `ColumnTransformer`

The different transformations were combined using `ColumnTransformer`.

`ColumnTransformer` is appropriate for datasets containing heterogeneous feature types because different transformations can be applied to different groups of columns within one preprocessing object.

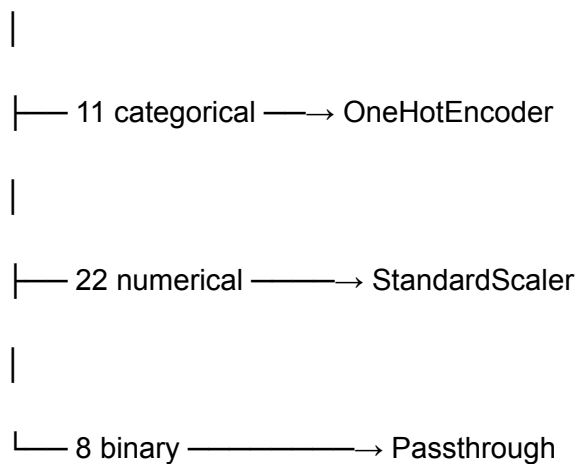
The baseline preprocessing structure is conceptually:

Original 36 features



The engineered preprocessing structure is:

Original 36 + 5 engineered features



Each preprocessor was fitted using only its corresponding training dataset.

5.10 Applying the Fitted Preprocessors

After fitting the preprocessing objects on the training data, the same fitted objects were used to transform the corresponding testing data.

The preprocessors were not fitted again on the test set.

This means that:

- categorical levels were learned from training data;
- numerical scaling parameters were learned from training data;
- the same learned transformations were applied to the testing data.

This preserves the independence of the test set and prevents preprocessing-related data leakage.

5.11 Processed Feature Representation

One-hot encoding increases the number of columns because a single categorical feature can be represented by multiple binary columns.

As a result, the original 36 baseline features were transformed into 244 model-ready features.

The engineered dataset contains five additional engineered features, resulting in 249 processed features.

The final shapes were:

Dataset	Original features	Processed features
Baseline training	36	244
Baseline testing	36	244
Engineered training	41	249

Engineered testing	41	249
--------------------	----	-----

The difference between the two processed representations is:

$$249 - 244 = 5 \text{ features}$$

This confirms that the five engineered features were retained through the preprocessing stage.

The expansion from 36 to 244 features in the baseline representation is primarily caused by one-hot encoding of the categorical variables. One-hot encoding does not represent 208 completely new pieces of information; rather, it represents the existing categorical information using multiple binary columns.

5.12 Preprocessing Validation

After transformation, the dimensions of the training and testing representations were checked.

The validation confirmed:

Baseline:

Training $\rightarrow 3,539 \times 244$

Testing $\rightarrow 885 \times 244$

Engineered:

Training $\rightarrow 3,539 \times 249$

Testing $\rightarrow 885 \times 249$

The matching feature counts between training and testing are important because a trained model must receive the same feature structure during training and prediction.

The validation therefore confirmed that:

1. the baseline training and testing data have identical processed feature dimensions;
2. the engineered training and testing data have identical processed feature dimensions;
3. the engineered representation contains exactly five additional processed features compared with the baseline representation.

5.13 Missing Values and Data Quality

Missing-value analysis performed during EDA found no missing values in the input features. Therefore, no imputation technique was required during preprocessing.

Duplicate analysis also found no duplicate feature rows requiring removal.

Additional data-quality checks were performed during EDA, including checks for invalid binary values, negative curricular-unit counts, implausible grades, and logical inconsistencies between curricular-unit variables.

Statistical outliers were investigated using the IQR method. These observations were not automatically removed because being statistically unusual does not necessarily mean that a value is erroneous. Where values represented plausible student observations, they were retained.

Therefore, preprocessing did not introduce unnecessary deletion or imputation operations.

5.14 Data Leakage Considerations

Data leakage was considered throughout the preparation process.

The prediction task is defined as predicting the student's final outcome using information available through the second semester. Therefore, second-semester academic variables and the engineered features derived from them are intentionally available to the model under the defined prediction scenario.

The main preprocessing-related leakage risk was allowing information from the test set to influence the learned transformations.

This was avoided by following the sequence:

Complete dataset



Train/test split



Separate baseline and engineered training sets



Fit preprocessing using training data only



Transform training data



Transform testing data using the same fitted rules

The testing data therefore remains unseen during the learning of the preprocessing parameters.

5.15 Final Preprocessing Result

The completed preprocessing stage produces two consistent model-ready datasets.

Dataset	Students	Original features	Processed features
Baseline training	3,539	36	244
Baseline testing	885	36	244

Engineered training	3,539	41	249
Engineered testing	885	41	249

The final preprocessing strategy can therefore be summarized as:

BASELINE

36 original features

↓

11 categorical → One-Hot Encoding

18 numerical → StandardScaler

7 binary → Passthrough

↓

244 processed features

and:

ENGINEERED

41 total features

↓

11 categorical → One-Hot Encoding

22 numerical → StandardScaler

8 binary → Passthrough

↓

249 processed features

The two representations are now ready for the modelling stage. The baseline representation provides the reference model, while the engineered representation allows the effect of the five engineered features to be evaluated objectively using the same modelling and evaluation procedure.

6. Machine Learning Model Development

After completing data preprocessing and feature engineering, the prepared dataset was used to develop and compare multiple classification models. The objective of this stage was not only to find a model that produced a high accuracy, but also to determine whether the engineered features improved model performance and to identify the most suitable model for the final application.

The prediction problem contains three possible outcomes:

- Dropout
- Enrolled
- Graduate

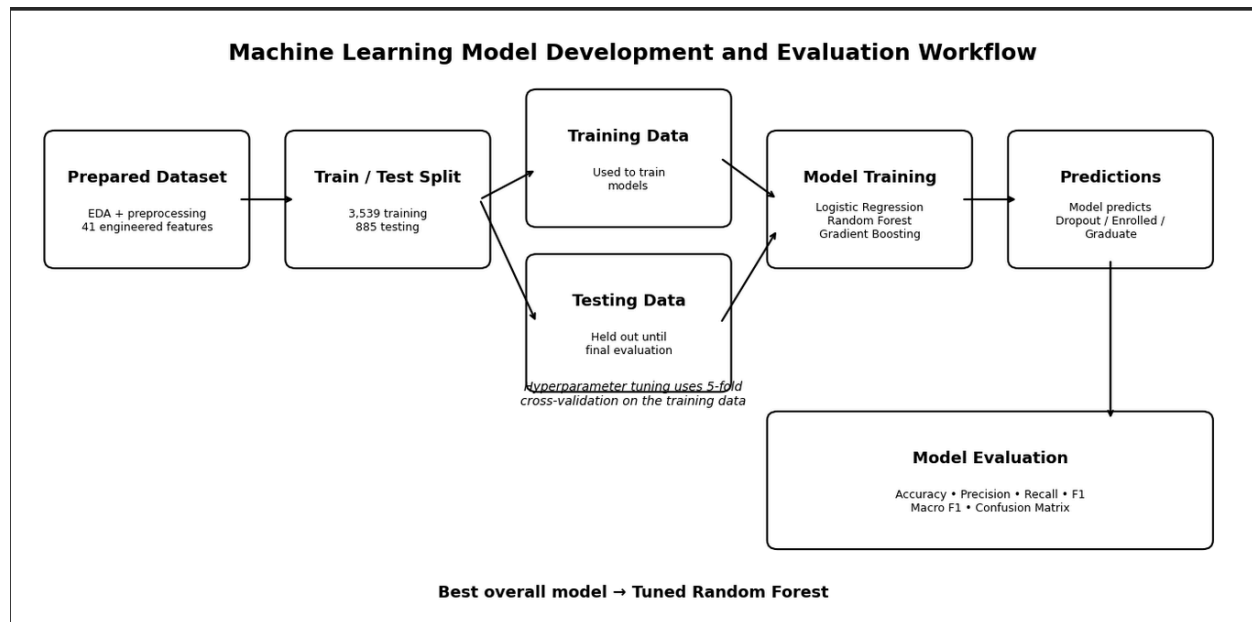
Therefore, the machine learning task is a multiclass classification problem.

Three model families were investigated:

1. Logistic Regression
2. Random Forest
3. Gradient Boosting

Logistic Regression was selected as a relatively simple classification model that provides a useful baseline. Random Forest was selected because it can model nonlinear relationships and interactions between features. Gradient Boosting was added as another tree-based ensemble method using a different learning strategy.

The models were evaluated using the same held-out test set so that their results could be compared under the same conditions.



6.1 Baseline and Engineered Datasets

An important part of the experiment was determining whether the feature engineering performed earlier actually improved predictive performance.

For this reason, two versions of the input data were prepared.

Baseline dataset

The baseline dataset contained only the original 36 input features from the UCI dataset.

The five newly created features were excluded from this dataset:

- `Sem1_approval_rate`

- Sem2_approval_rate
- Grade_change
- Financial_stress
- Approval_rate_change

The baseline dataset therefore represented the model's performance using the original information without the additional engineered features.

Engineered dataset

The engineered dataset contained the original 36 features together with the five newly created features.

Therefore:

Dataset	Original features	Engineered features	Total raw features
Baseline	36	0	36
Engineered	36	5	41

After preprocessing, the difference in feature dimensions was:

Dataset	Processed training features	Processed testing features
Baseline	244	244

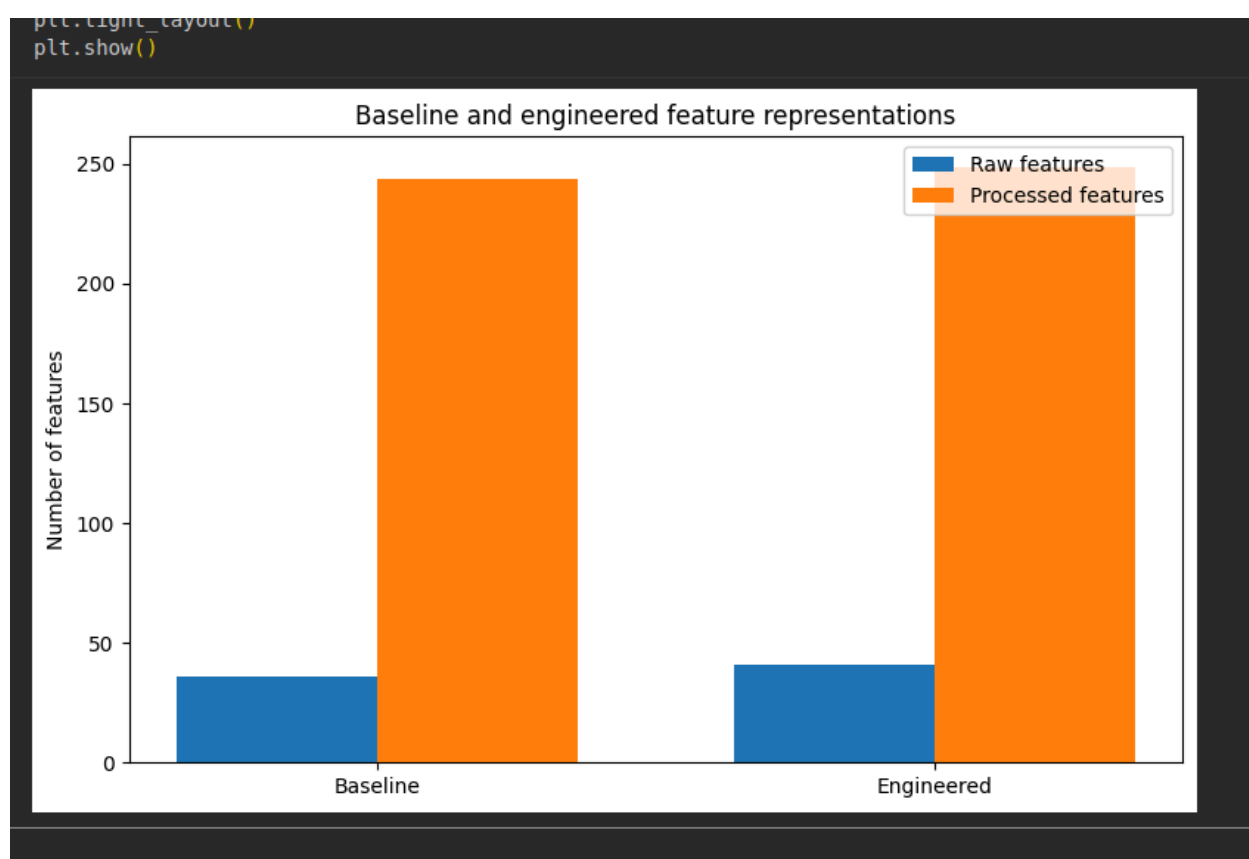
Engineered	249	249
------------	-----	-----

The additional five processed columns confirm that the five engineered features were retained by the preprocessing stage.

This baseline-versus-engineered design is important because simply reporting the final model's accuracy would not show whether the feature engineering itself provided any benefit.

The comparison can therefore answer two separate questions:

1. How well can the models perform using the original features?
2. Does adding the engineered features improve their performance?



7. Logistic Regression

7.1 Purpose of Logistic Regression

Logistic Regression was used as the first classification model.

Despite its name, Logistic Regression is a classification algorithm rather than a conventional regression model. The model learns a relationship between the input features and the probability of belonging to each target class.

For this project, the input contains three classes: Dropout, Enrolled, and Graduate. The model therefore produces a probability distribution across these classes and selects the class with the highest predicted probability.

The processed feature values are used as the model inputs. The model learns coefficients that determine how strongly the different input features contribute to the prediction.

The model was configured with:

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(
```

```
    max_iter=1000,
```

```
    random_state=42
```

```
)
```

The `max_iter=1000` setting gives the optimization procedure enough iterations to reach convergence with the processed feature set. It represents the maximum number of optimization iterations rather than a fixed number of iterations that must always be performed.

7.2 Logistic Regression Using the Baseline Dataset

The first experiment used the original 36 features without the five engineered features.

The model was trained using the processed baseline training data and then evaluated using the untouched baseline test data.

The baseline Logistic Regression model achieved:

Accuracy: 76.84%

The classification results were:

Class	Precision	Recall	F1-score	Support
Dropout	0.79	0.75	0.77	284
Enrolled	0.55	0.40	0.46	159
Graduate	0.81	0.91	0.86	442
Accuracy			0.77	885
Macro average	0.72	0.69	0.70	885
Weighted average	0.76	0.77	0.76	885

The model performed particularly well for the Graduate class, with an F1-score of 0.86 and recall of 0.91.

The Enrolled class was considerably more difficult. Its recall was only 0.40, meaning that the model correctly identified 40% of the students who were actually Enrolled.

This shows an important limitation of evaluating the model using accuracy alone. Although the overall accuracy was 76.84%, the model was much less successful at recognizing the smaller Enrolled class.

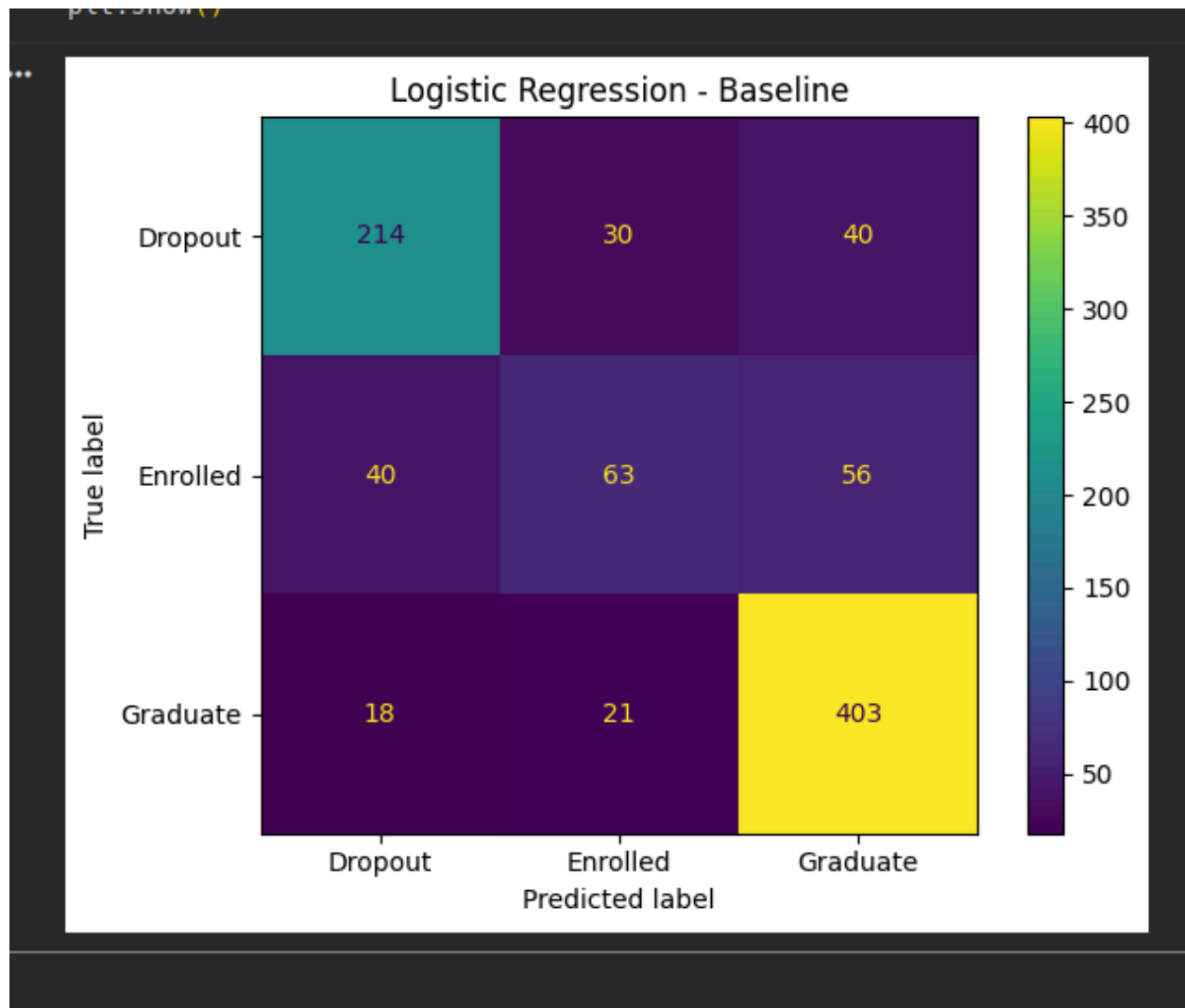
Confusion matrix: baseline Logistic Regression

The confusion matrix was:

	Predicted		
	Dropout	Enrolled	Graduate
Actual Dropout	214	30	40
Actual Enrolled	40	63	56
Actual Graduate	18	21	403

The diagonal values represent correct predictions.

For example, 214 of the 284 actual Dropout students were correctly classified as Dropout, while 63 of the 159 actual Enrolled students were correctly classified as Enrolled.



7.3 Logistic Regression Using the Engineered Dataset

The second Logistic Regression experiment used the engineered dataset containing the original 36 features plus the five engineered features.

The model configuration remained the same so that the comparison would isolate the effect of feature engineering rather than changes to the algorithm.

The engineered Logistic Regression model achieved:

Accuracy: 77.40%

The classification results were:

Class	Precision	Recall	F1-score	Support
Dropout	0.78	0.77	0.78	284
Enrolled	0.58	0.39	0.47	159
Graduate	0.81	0.91	0.86	442
Accuracy			0.77	885
Macro average	0.72	0.69	0.70	885
Weighted average	0.76	0.77	0.76	885

Compared with the baseline Logistic Regression model, accuracy increased:

76.84% → 77.40%

This is an improvement of approximately **0.56 percentage points**.

The Dropout F1-score increased from 0.77 to 0.78 and the Enrolled F1-score increased from 0.46 to 0.47.

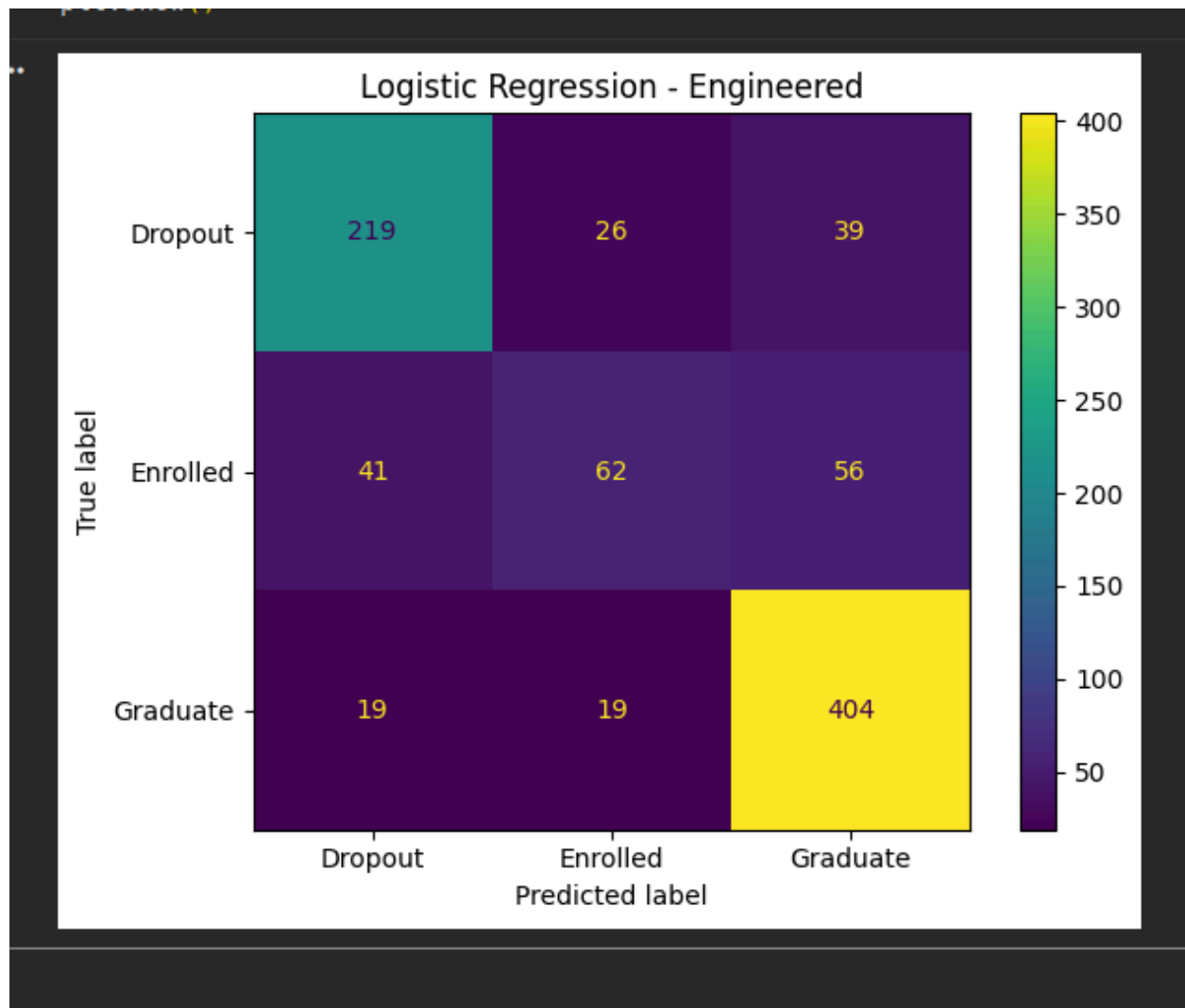
However, Enrolled recall changed slightly from 0.40 to 0.39. Therefore, the improvement did not mean that the model became substantially better at identifying every Enrolled student. Instead, the model gained a small overall improvement, while the Enrolled class remained difficult.

Confusion matrix: engineered Logistic Regression

Predicted

Dropout Enrolled Graduate

Actual Dropout	219	26	39
Actual Enrolled	41	62	56
Actual Graduate	19	19	404



The comparison between the two Logistic Regression experiments demonstrates that the engineered features provided a small but measurable improvement in overall predictive performance.

7.4 Logistic Regression Hyperparameter Tuning

After evaluating the baseline and engineered Logistic Regression models, hyperparameter tuning was performed on the engineered model.

The regularization parameter C was tested using the following values:

$C = [0.01, 0.1, 1, 10, 100]$

The parameter C controls the strength of regularization. Smaller values apply stronger regularization, while larger values allow the model coefficients greater freedom.

Five-fold cross-validation was used, and **macro F1** was selected as the scoring metric because it gives equal importance to the three target classes.

The best parameter was:

$C = 1$

The best cross-validation macro F1-score was:

0.7111

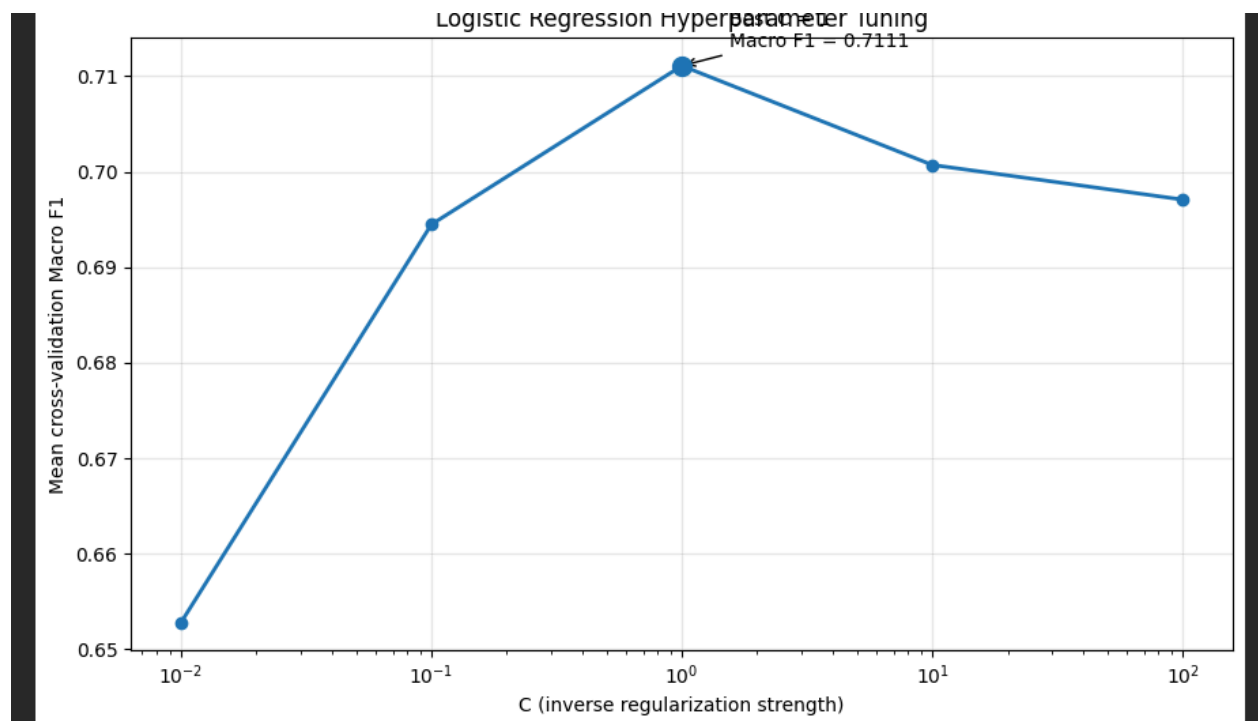
The tuned model produced the same test performance as the original engineered Logistic Regression model:

- Accuracy: **77.40%**
- Macro F1: **0.70**

This occurred because the best value found during tuning, $C = 1$, was already the parameter value used by the original Logistic Regression model.

Therefore, the tuning process confirmed that none of the alternative values tested produced a better result on the cross-validation data.

This is a useful result rather than a failed experiment. It shows that the original regularization setting was already suitable within the tested search range.



8. Random Forest

8.1 Purpose of Random Forest

Random Forest was selected as the second major model because it provides a fundamentally different modelling approach from Logistic Regression.

Instead of learning one linear relationship, Random Forest combines multiple decision trees. Each tree is trained using sampled data and feature subsets, and the individual tree predictions are combined to obtain the final classification. This allows the model to capture nonlinear relationships and interactions that may not be represented well by a linear classifier.

The initial Random Forest model was configured with 100 trees:

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(
```

```
n_estimators=100,  
  
random_state=42  
)
```

The `n_estimators` parameter specifies the number of trees in the forest.

The `random_state=42` setting was used to make the experiment reproducible.

8.2 Random Forest Using the Baseline Dataset

The baseline Random Forest was trained using only the original 36 features.

The model achieved:

Accuracy: 76.16%

The classification results were:

Class	Precision	Recall	F1-score	Support
Dropout	0.77	0.76	0.77	284
Enrolled	0.56	0.25	0.34	159
Graduate	0.78	0.95	0.86	442

Accuracy			0.76	885
Macro average	0.70	0.65	0.66	885
Weighted average	0.74	0.76	0.74	885

The baseline Random Forest achieved slightly lower accuracy than baseline Logistic Regression:

76.16% versus 76.84%

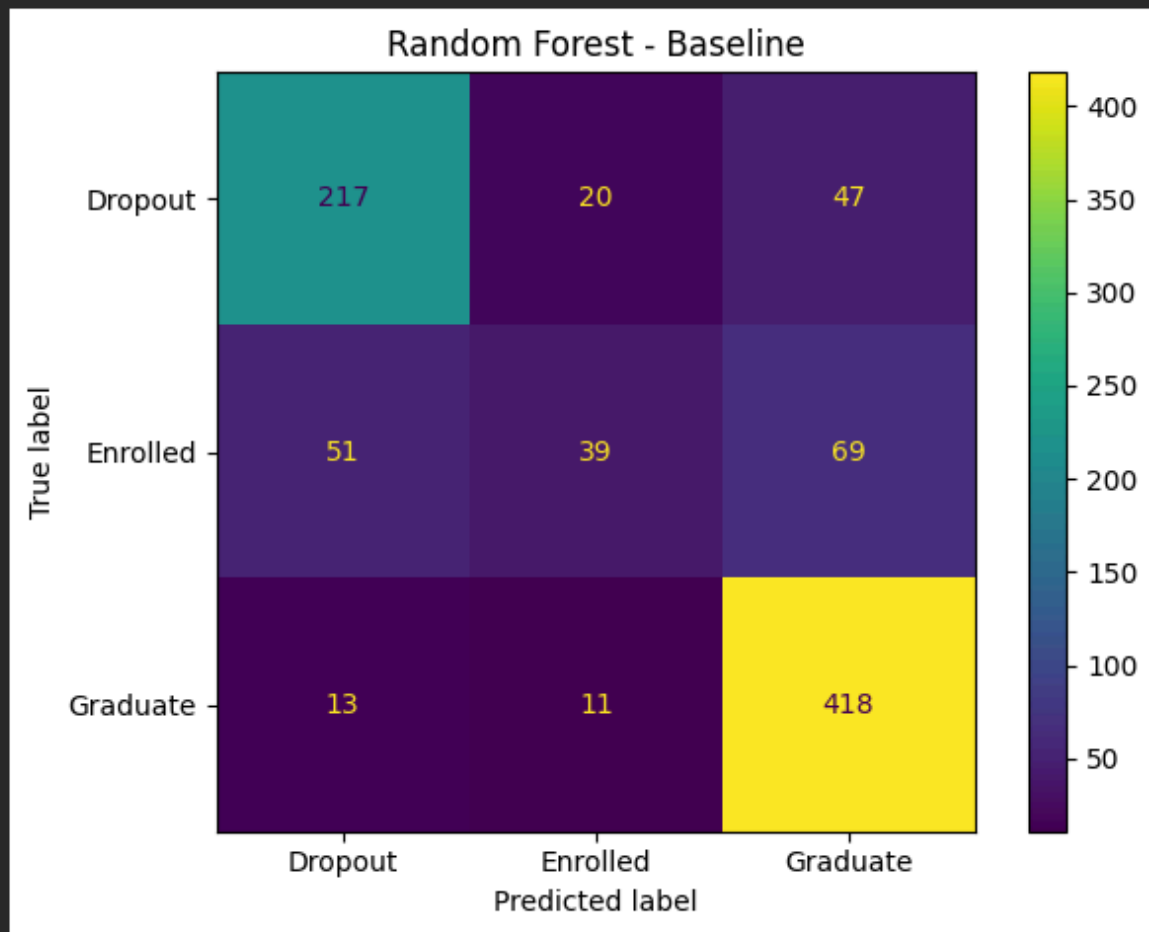
The most significant weakness was the Enrolled class.

Its recall was only **0.25**, meaning the model correctly identified only one quarter of the actual Enrolled students.

The Graduate class was predicted very strongly, with recall of 0.95 and an F1-score of 0.86.

This result illustrates that a tree-based ensemble is not automatically superior simply because it is more complex.

```
plt.tight_layout()
plt.show()
```



8.3 Random Forest Using the Engineered Dataset

The second Random Forest experiment used the engineered dataset containing 41 raw features.

The engineered Random Forest achieved:

Accuracy: 77.85%

The classification results were:

Class	Precision	Recall	F1-score	Support
Dropout	0.80	0.79	0.79	284
Enrolled	0.58	0.33	0.42	159
Graduate	0.80	0.93	0.86	442
Accuracy			0.78	885
Macro average	0.73	0.68	0.69	885
Weighted average	0.76	0.78	0.76	885

Compared with the baseline Random Forest:

76.16% → 77.85%

This is an improvement of approximately **1.69 percentage points**.

This improvement was larger than the corresponding improvement observed for Logistic Regression.

The Dropout F1-score increased from 0.77 to 0.79, while Enrolled F1-score increased from 0.34 to 0.42.

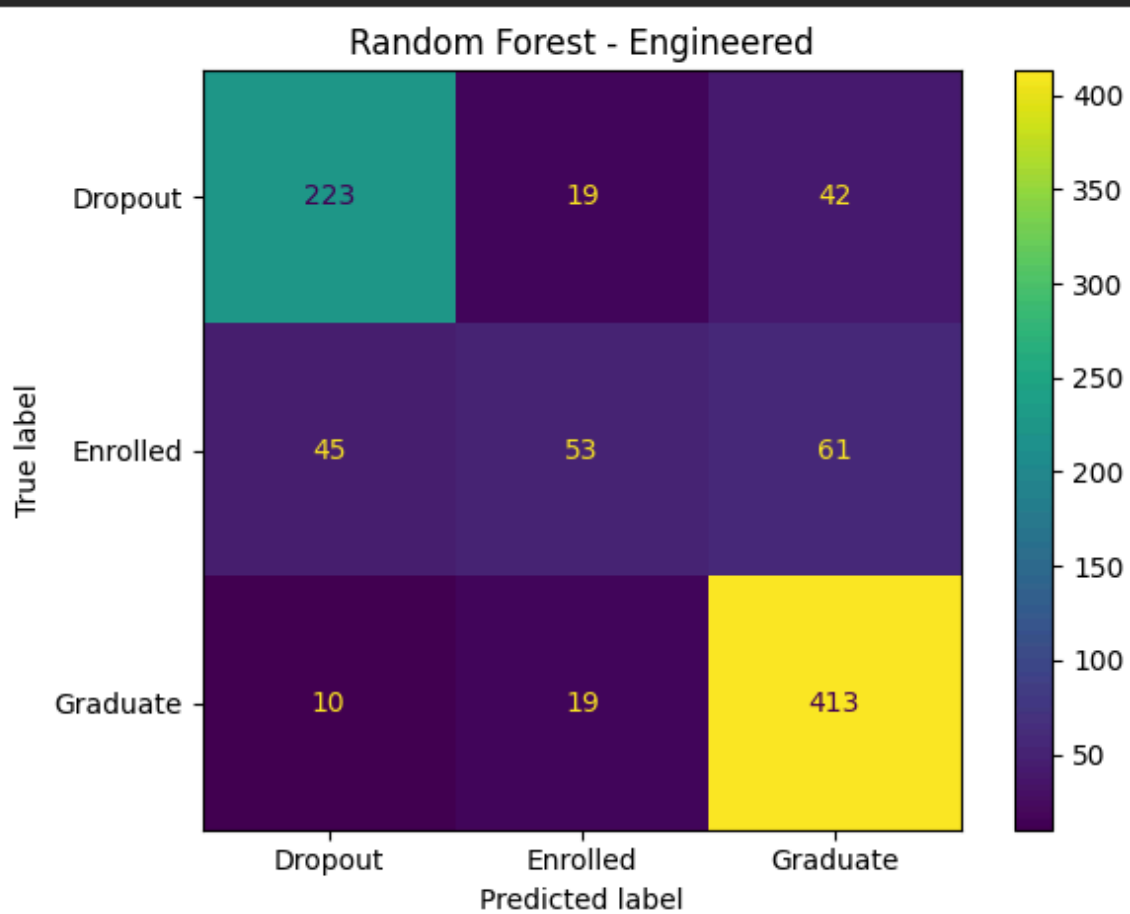
Therefore, the engineered features provided a more noticeable benefit to Random Forest than they did to Logistic Regression.

However, Enrolled recall remained low at 0.33.

Confusion matrix: engineered Random Forest

	Predicted		
	Dropout	Enrolled	Graduate
Actual Dropout	223	19	42
Actual Enrolled	45	53	61
Actual Graduate	10	19	413

```
plt.title("Random Forest - Engineered")
plt.tight_layout()
plt.show()
```



8.4 Random Forest Hyperparameter Tuning

After comparing the baseline and engineered Random Forest models, hyperparameter tuning was performed using the engineered dataset.

The following combinations were tested:

```
rf_param_grid = {
```

```
    "n_estimators": [100, 200],
```

```
    "max_depth": [None, 10, 20],
```

```
"min_samples_split": [2, 5],  
  
"min_samples_leaf": [1, 2]  
  
}
```

This produced:

24 parameter combinations × 5 folds = 120 model fits

Macro F1 was again used as the scoring metric.

The best parameter combination was:

n_estimators = 200

max_depth = None

min_samples_split = 2

min_samples_leaf = 1

The best cross-validation macro F1-score was:

0.6952

The tuned Random Forest achieved:

Accuracy: 78.08%

This was an improvement over the engineered Random Forest:

77.85% → 78.08%

The improvement was relatively small, but the tuned model became the strongest overall model based on the final test results.

The final classification results were:

Class	Precision	Recall	F1-score	Support
Dropout	0.81	0.79	0.80	284
Enrolled	0.60	0.33	0.43	159
Graduate	0.80	0.94	0.86	442
Accuracy			0.78	885
Macro average	0.74	0.69	0.70	885
Weighted average	0.77	0.78	0.76	885

Confusion matrix: tuned Random Forest

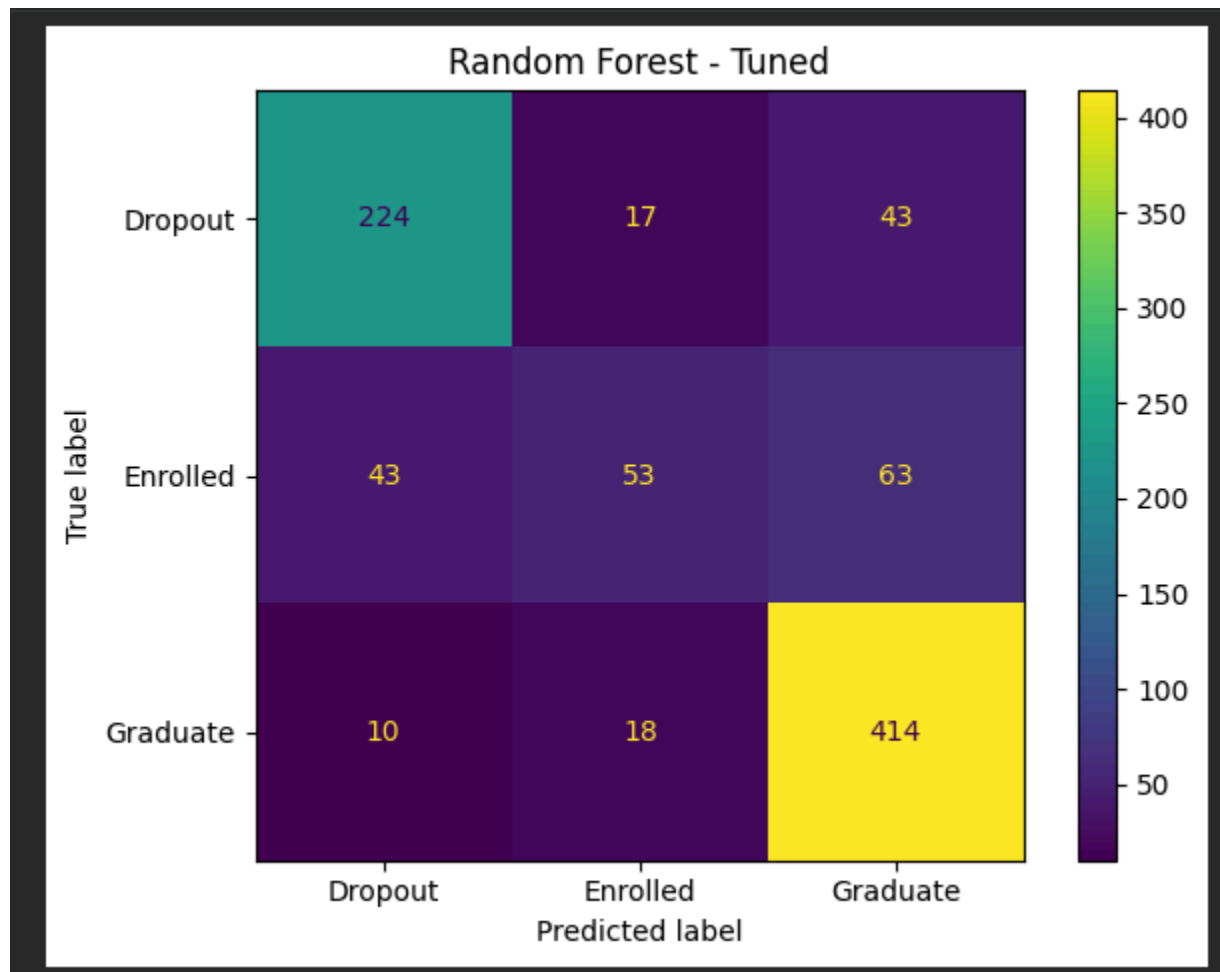
Predicted

Dropout Enrolled Graduate

Actual Dropout 224 17 43

Actual Enrolled 43 53 63

Actual Graduate 10 18 414



The tuned Random Forest correctly identified 224 of 284 Dropout students and 414 of 442 Graduate students.

However, only 53 of the 159 actual Enrolled students were correctly identified.

Therefore:

- Dropout recall = $224 / 284$ = approximately **78.9%**
- Enrolled recall = $53 / 159$ = approximately **33.3%**
- Graduate recall = $414 / 442$ = approximately **93.7%**

The model is therefore particularly strong at identifying Graduate students but considerably weaker at identifying Enrolled students.

9. Gradient Boosting

9.1 Purpose of Gradient Boosting

Gradient Boosting was included as an additional model family to provide another tree-based approach for comparison.

Unlike Random Forest, where many trees are constructed independently and their predictions are combined, Gradient Boosting builds the model sequentially. Each stage attempts to improve upon the errors made by previous stages.

This gives the project three different modelling approaches:

Model	Main idea
Logistic Regression	Learns a classification relationship using model coefficients
Random Forest	Combines many independently constructed decision trees
Gradient Boosting	Builds trees sequentially to improve previous errors

9.2 Engineered Gradient Boosting

Gradient Boosting was trained using the engineered dataset containing the five additional features.

The initial model achieved:

Accuracy: 75.37%

Class	Precision	Recall	F1-score	Support
Dropout	0.77	0.74	0.75	284
Enrolled	0.48	0.36	0.41	159
Graduate	0.81	0.90	0.86	442
Accuracy			0.75	885
Macro average	0.69	0.67	0.67	885
Weighted average	0.74	0.75	0.74	885

The model again performed best on the Graduate class and struggled with the Enrolled class.

However, its overall performance was below both Logistic Regression and Random Forest.

9.3 Gradient Boosting Hyperparameter Tuning

The following parameter grid was tested:

```
gb_param_grid = {  
    "n_estimators": [100, 200],  
    "learning_rate": [0.05, 0.1],  
    "max_depth": [2, 3],  
    "min_samples_split": [2, 5],  
    "min_samples_leaf": [1, 2]  
}
```

This produced:

32 parameter combinations × 5 folds = 160 model fits

The best configuration was:

n_estimators = 200

learning_rate = 0.1

max_depth = 3

min_samples_split = 5

min_samples_leaf = 2

The tuned model achieved:

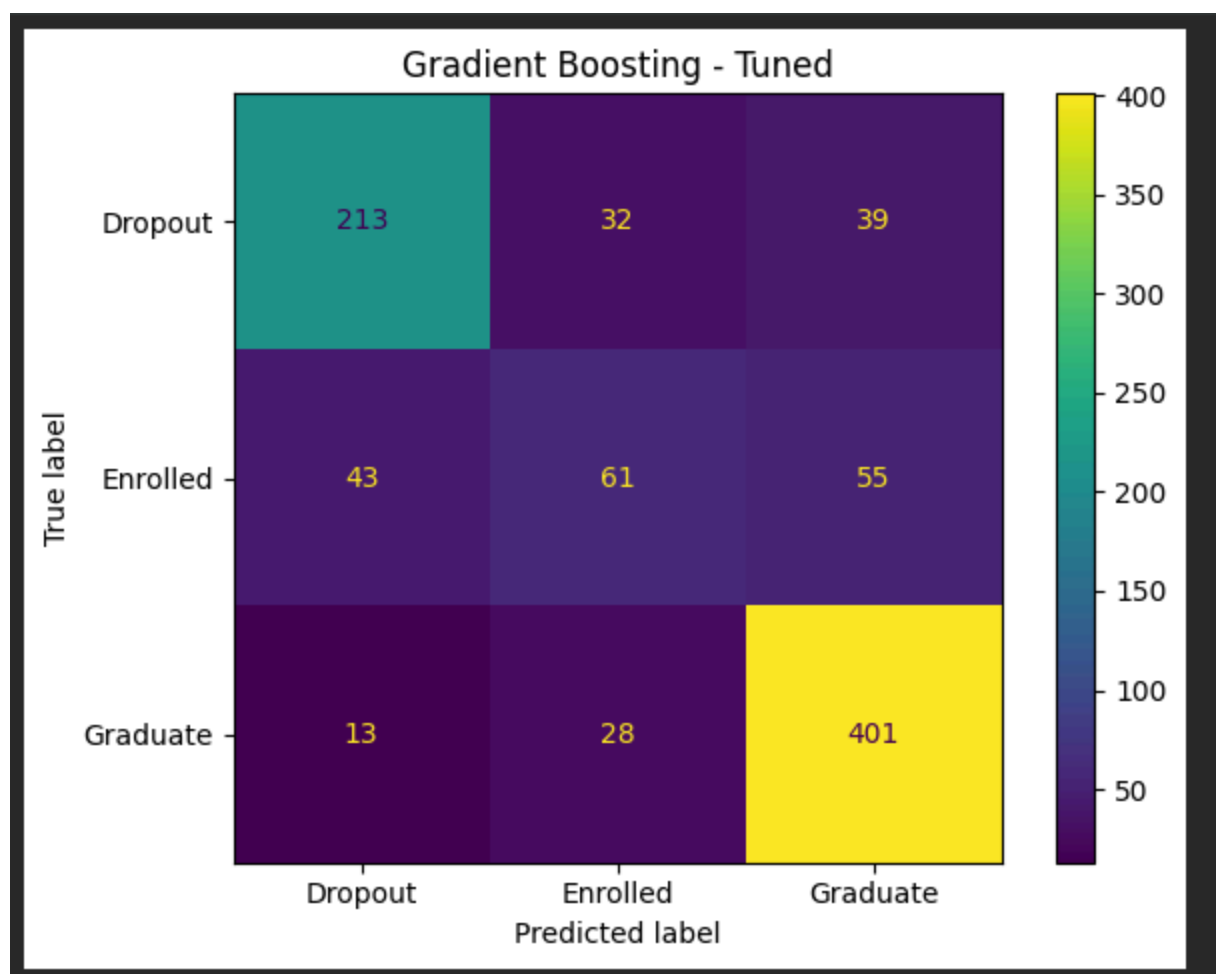
Accuracy: 76.27%

The macro F1-score increased from **0.67 to 0.69**.

Therefore, hyperparameter tuning improved Gradient Boosting, but it remained below Logistic Regression and Random Forest in overall performance.

The tuned classification results were:

Class	Precision	Recall	F1-score	Support
Dropout	0.79	0.75	0.77	284
Enrolled	0.50	0.38	0.44	159
Graduate	0.81	0.91	0.86	442
Accuracy			0.76	885
Macro average	0.70	0.68	0.69	885
Weighted average	0.75	0.76	0.75	885



The tuned Gradient Boosting confusion matrix was:

Predicted

Dropout Enrolled Graduate

Actual Dropout 213 32 39

Actual Enrolled 43 61 55

Actual Graduate 13 28 401

10. Understanding the Evaluation Metrics

Because the dataset contains three classes with different numbers of observations, multiple evaluation metrics were used rather than relying only on accuracy.

10.1 Precision

Precision answers the question:

When the model predicts a student belongs to a particular class, how often is that prediction correct?

For example, if the model predicts a group of students as Enrolled, precision measures how many of those predicted Enrolled students are actually Enrolled.

10.2 Recall

Recall answers the question:

Of all students who actually belong to a particular class, how many did the model successfully identify?

This is particularly important in this project because identifying actual Dropout, Enrolled, and Graduate students has different implications.

For example, the tuned Random Forest achieved an Enrolled recall of approximately 0.33. This means that only about one third of the actual Enrolled students were identified as Enrolled by the model.

10.3 F1-score

F1-score combines precision and recall using their harmonic mean:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A high F1-score requires both precision and recall to be reasonably strong.

10.4 Macro F1-score

Macro F1 calculates the F1-score separately for each class and then gives every class equal weight when calculating the average.

For this project:

$$\text{Macro F1} = (\text{F1 Dropout} + \text{F1 Enrolled} + \text{F1 Graduate}) / 3$$

This means the Graduate class does not receive more weight simply because it contains more students.

Macro F1 was therefore useful because the Enrolled class is smaller and harder to classify.

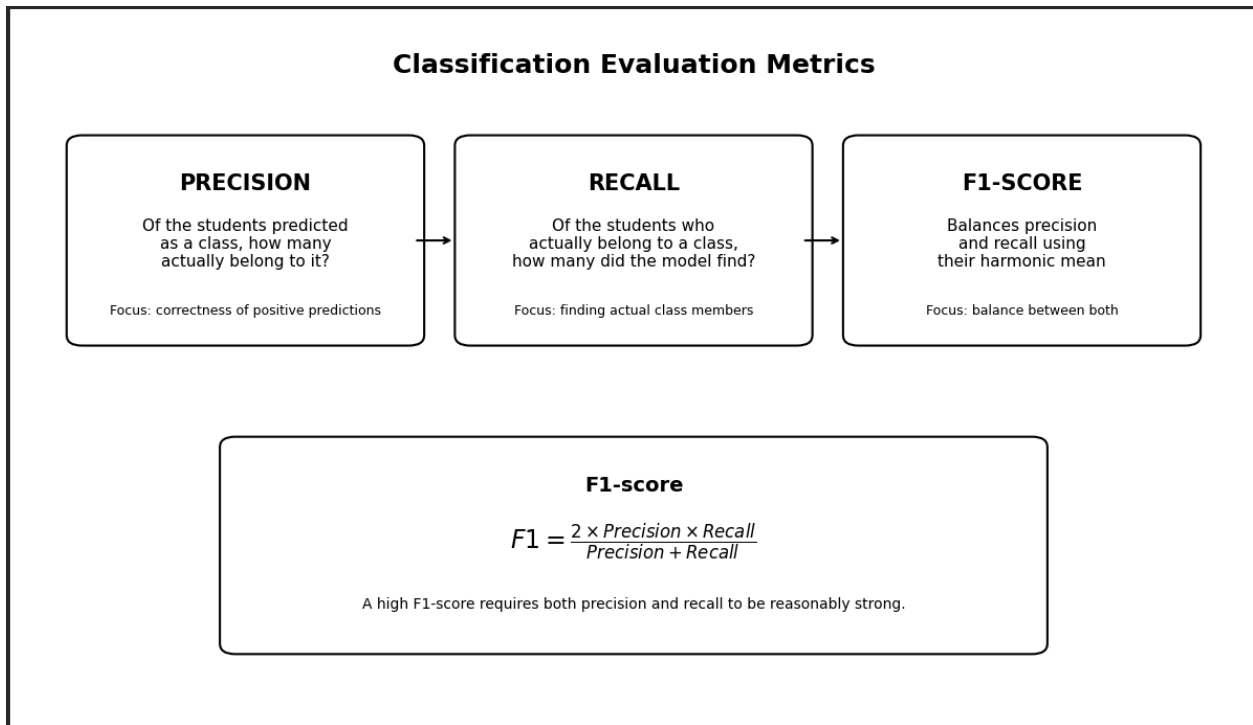
Scikit-learn's classification metrics distinguish the macro average from the weighted average: the macro average treats classes equally, while the weighted average accounts for the number of observations in each class.

10.5 Weighted F1-score

Weighted F1 calculates each class's F1-score while giving greater weight to classes with more observations.

This means the large Graduate class has a greater influence on the weighted F1-score than the smaller Enrolled class.

For this reason, macro F1 was considered alongside weighted F1 rather than relying on the weighted value alone.

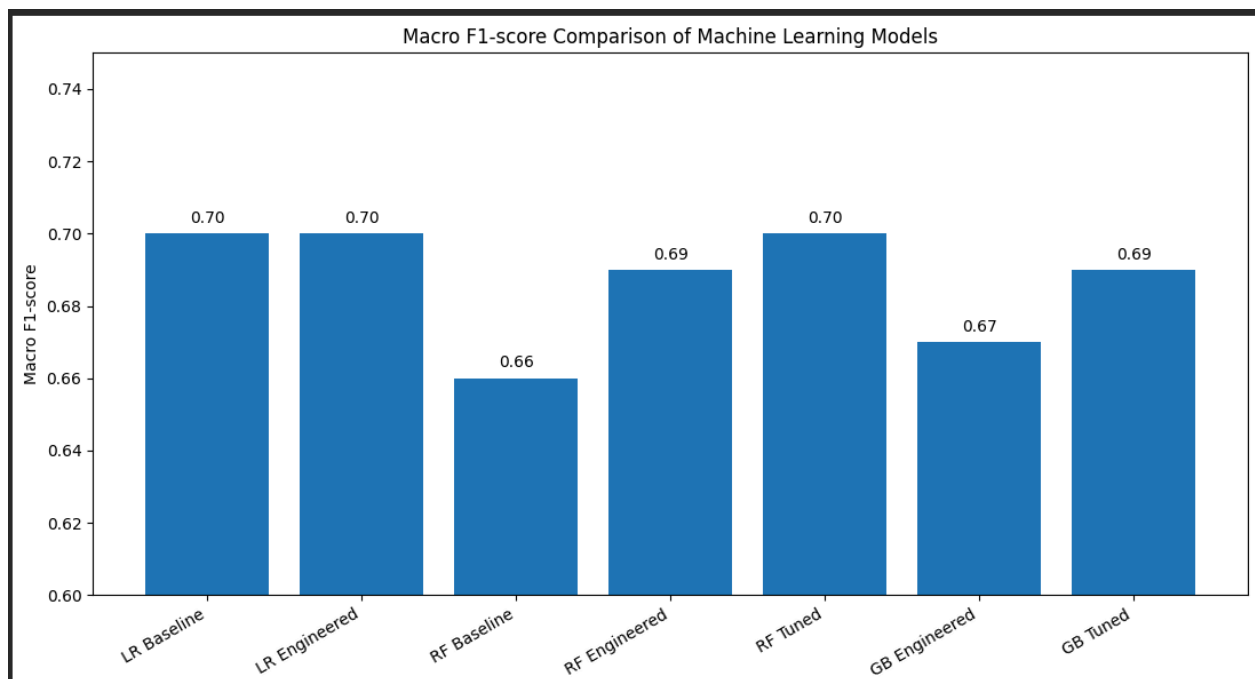
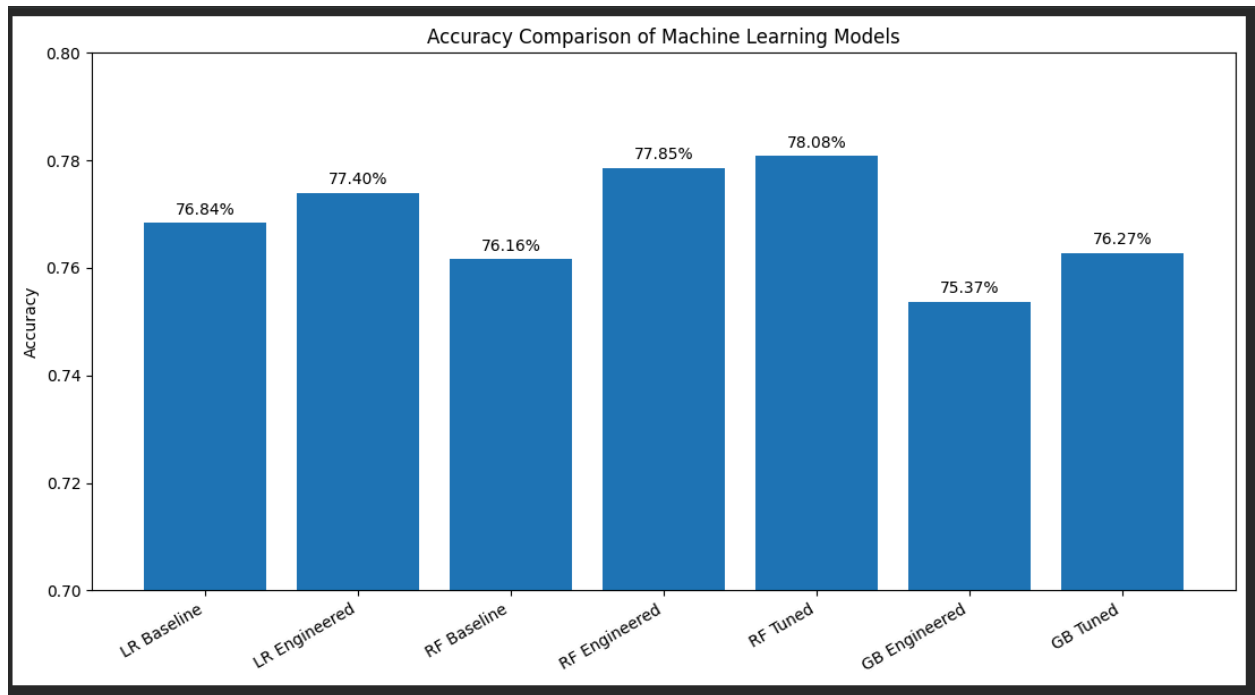


11. Overall Model Comparison

The results from all experiments are summarized below.

Model	Accuracy	Macro F1	Dropout F1	Enrolled F1	Graduate F1
Logistic Regression – Baseline	0.768362	0.70	0.77	0.46	0.86

Logistic Regression – Engineered	0.774011	0.70	0.78	0.47	0.86
Random Forest – Baseline	0.761582	0.66	0.77	0.34	0.86
Random Forest – Engineered	0.778531	0.69	0.79	0.42	0.86
Random Forest – Tuned	0.780791	0.70	0.80	0.43	0.86
Gradient Boosting – Engineered	0.753672	0.67	0.75	0.41	0.86
Gradient Boosting – Tuned	0.762712	0.69	0.77	0.44	0.86



The comparison produces several important findings.

11.1 Effect of Feature Engineering

Feature engineering improved both models for which a direct baseline-versus-engineered comparison was performed.

For Logistic Regression:

76.84% → 77.40%

For Random Forest:

76.16% → 77.85%

Therefore, the engineered features provided measurable predictive value.

The improvement was particularly noticeable for Random Forest, where the overall accuracy increased by approximately 1.69 percentage points.

11.2 Effect of Hyperparameter Tuning

Hyperparameter tuning produced modest improvements.

For Random Forest:

77.85% → 78.08%

For Gradient Boosting:

75.37% → 76.27%

For Logistic Regression, tuning selected $C = 1$, which was already the original value, so the test performance remained unchanged at 77.40%.

These results demonstrate that tuning does not necessarily produce a large performance increase. Its purpose is to systematically search for a better configuration rather than guarantee an improvement.

11.3 Difficulty of the Enrolled Class

Across all models, Graduate was consistently the easiest class to identify.

Enrolled was consistently the most difficult.

This pattern is visible in both the F1-scores and the confusion matrices.

The difficulty is important because accuracy can remain relatively high even when a model performs poorly on the smaller Enrolled class.

12. Confusion Matrix Comparison

The confusion matrices provide a more detailed view of the final models.

The three final candidates were:

- Engineered Logistic Regression
- Tuned Random Forest
- Tuned Gradient Boosting

The results show that the models make a similar type of error: many actual Enrolled students are classified as either Dropout or Graduate.

For Logistic Regression:

Actual Enrolled:

41 → Dropout

62 → Enrolled

56 → Graduate

For Tuned Random Forest:

Actual Enrolled:

43 → Dropout

53 → Enrolled

63 → Graduate

For Tuned Gradient Boosting:

Actual Enrolled:

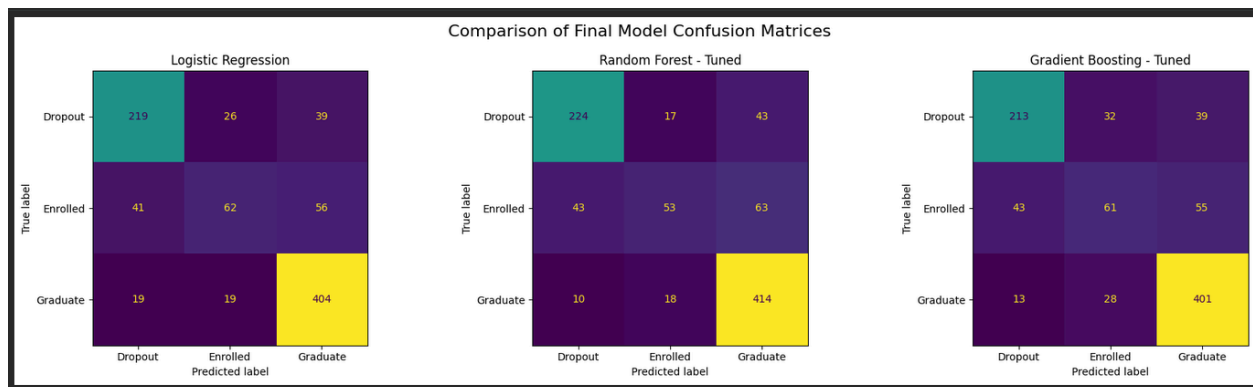
43 → Dropout

61 → Enrolled

55 → Graduate

The tuned Random Forest achieves the strongest overall accuracy partly because it performs particularly well on the larger Graduate class and also improves the Dropout class.

However, Logistic Regression and Gradient Boosting identify more actual Enrolled students than the tuned Random Forest.



This figure should be placed directly after the preceding discussion because it allows the reader to visually verify the classification patterns being described.

13. Feature Importance of the Final Random Forest

After selecting the Random Forest as the final model, feature importance was examined to understand which processed features contributed most strongly to the tree-based model.

The final Random Forest produced 249 feature-importance values corresponding to the 249 processed features.

The highest-ranking features were:

Rank	Feature	Importance
1	Sem2 approval rate	0.095201
2	Sem1 approval rate	0.068842
3	Curricular units 2nd sem (approved)	0.066075
4	Curricular units 2nd sem (grade)	0.055292
5	Curricular units 1st sem (approved)	0.052573
6	Curricular units 1st sem (grade)	0.045734

7	Approval rate change	0.031513
8	Grade change	0.029861
9	Admission grade	0.028240
10	Age at enrollment	0.027479
11	Previous qualification (grade)	0.027069
12	Tuition fees up to date	0.026806
13	Curricular units 2nd sem (evaluations)	0.025772
14	Curricular units 1st sem (evaluations)	0.025465
15	Unemployment rate	0.017524
16	GDP	0.016974

17	Inflation rate	0.016185
18	Curricular units 2nd sem (enrolled)	0.015981
19	Curricular units 1st sem (enrolled)	0.015317
20	Scholarship holder	0.012246

The most important feature was `Sem2_approval_rate`, followed by `Sem1_approval_rate`.

This provides additional evidence that academic progress during the two semesters is strongly associated with the final classification.

The engineered features also appear prominently in the importance ranking.

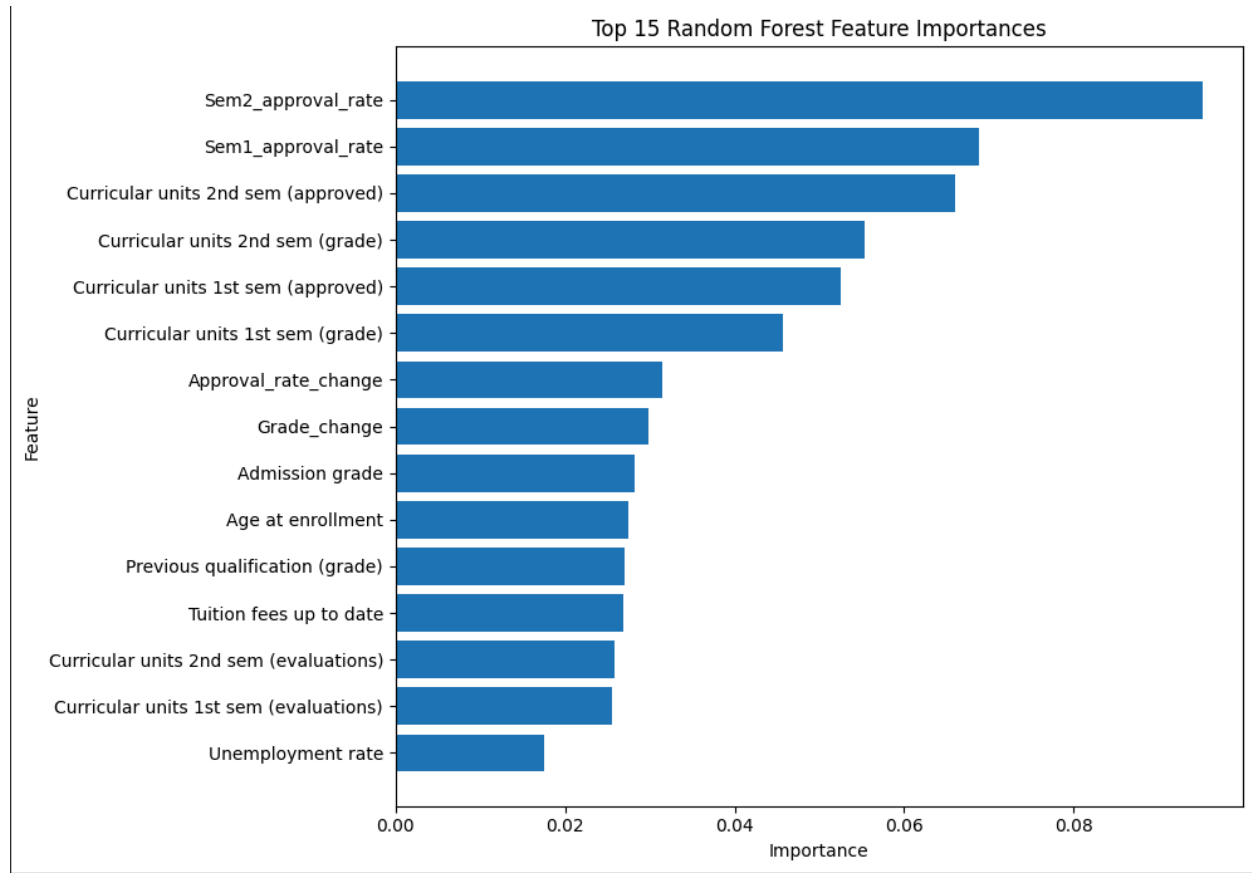
`Approval_rate_change` ranked seventh and `Grade_change` ranked eighth.

This is consistent with the earlier feature-engineering rationale: instead of considering semester performance only as separate raw values, the engineered features also describe approval efficiency and change in academic performance between semesters.

However, these values are **model-specific feature-importance measures and should not be interpreted as causal relationships**. A feature being important to the Random Forest does not prove that changing that feature would directly cause a change in the student's outcome.

The Random Forest documentation identifies `feature_importances_` as impurity-based feature importance and notes that such importance measures can be misleading for features with many unique values.

A permutation-importance analysis was investigated during the notebook work but was not included in the final analysis after a sparse-matrix compatibility issue with the installed environment. Therefore, the feature-importance discussion in this report is explicitly based on the Random Forest's impurity-based importance values.



The bar chart should be placed immediately after the feature-importance table.

14. Final Model Selection

Based on the complete evaluation, the **Tuned Random Forest Classifier** was selected as the final machine learning model.

Its final test performance was:

- Accuracy: **78.08%**
- Macro F1: **0.70**
- Dropout F1: **0.80**
- Enrolled F1: **0.43**
- Graduate F1: **0.86**

The model achieved the highest overall test accuracy among all evaluated experiments.

The selection was not based on accuracy alone. The model was compared using class-level precision, recall, F1-score, macro F1-score, and confusion matrices.

The tuned Random Forest provided the strongest overall balance, especially for the Dropout and Graduate classes.

However, Logistic Regression achieved a higher Enrolled F1-score:

- Logistic Regression: **0.47**
- Tuned Random Forest: **0.43**

Therefore, the selection involves a trade-off.

If the primary objective were specifically to maximize identification of the Enrolled class, Logistic Regression would be a stronger candidate.

For the overall project objective, the tuned Random Forest was selected because it achieved the highest overall accuracy while maintaining a competitive macro F1-score.

The final model configuration was:

```
RandomForestClassifier(  
  
    n_estimators=200,  
  
    max_depth=None,  
  
    min_samples_split=2,  
  
    min_samples_leaf=1,  
  
    random_state=42  
)
```

The final selected model will be used for the application integration stage.

15. Machine Learning Limitations

Although the final model achieved an accuracy of 78.08%, several limitations should be considered.

The first limitation is the difficulty of the Enrolled class. The tuned Random Forest correctly identified only approximately 33% of actual Enrolled students. Therefore, the model should not be interpreted as equally reliable for all three outcomes.

The second limitation is the class distribution. Graduate is the largest class and Enrolled is the smallest. The target distribution is therefore moderately imbalanced rather than perfectly balanced. This is another reason why accuracy alone was not used for model selection.

The third limitation is that Random Forest feature importance does not establish causation. The importance values describe how the trained model uses the input features, not why a student ultimately experiences a particular outcome.

Finally, the prediction point for this project must remain clearly defined. The model uses second-semester academic information because the project defines prediction as occurring after the second semester. Therefore, these variables are valid inputs within this project's stated prediction scenario. They would not be valid for a model that was intended to predict dropout at the time of initial enrollment.

16. Machine Learning Stage Summary

The model-development stage compared the effect of feature engineering and algorithm selection systematically.

The original 36-feature dataset was first used to establish baseline performance. The five engineered features were then added to create a 41-feature engineered dataset, allowing the effect of feature engineering to be measured directly.

The results showed that:

- Logistic Regression improved from **76.84% to 77.40%** after feature engineering.
- Random Forest improved from **76.16% to 77.85%** after feature engineering.
- Random Forest improved further from **77.85% to 78.08%** after hyperparameter tuning.
- Gradient Boosting improved from **75.37% to 76.27%** after hyperparameter tuning.

- Graduate was consistently the easiest class to predict.
- Enrolled was consistently the most difficult class to predict.
- Semester-performance features were the most influential features in the final Random Forest.
- The final selected model was the **Tuned Random Forest Classifier** with **78.08% accuracy and 0.70 macro F1-score**.

The selected model is now ready to be integrated into the application's machine learning prediction service.